

ArchLinker: An LLM Workflow for Software Architectural Traceability Link Recovery

ANONYMOUS AUTHOR(S)

Trace links between software architecture artifacts and code support impact analysis, change propagation, and compliance review, yet they are the hardest to recover: the artifacts describe the system on a higher level of abstraction and rarely directly match the names in the code. Lexical recovery overlooks aliases and implicit references, while a per-sentence Large Language Model (LLM) rebuilds that knowledge on every call and accepts plausible-but-wrong links. We present ArchLinker, a multi-stage LLM workflow for architecture document-architecture model traceability link recovery that keeps shared alias knowledge separate from two recovery routes. The first stage derives an alias table over the document and component set. The two routes then generate candidates from written names or references to earlier sentences, derive evidence for each candidate, and apply a route-specific judge. One judge handles all written-name forms, while another judge handles resolved references. On five open-source benchmark projects, we evaluate ArchLinker against the state of the art. ArchLinker improves performance in comparison to the strongest baseline, run on the same LLM backend, by 13 percentage points (pp) in architecture doc-model average F_1 , and by 13 pp in the recall-weighted average F_2 . This gain carries through to 7 pp of average F_1 on the downstream doc-code task. Under an architecture-driven suite that adds per-component measures, the gap widens to 30 and 28 pp. A replication package provides prompts, code, raw LLM responses, and per-project results.

Additional Key Words and Phrases: Documentation, large language models, natural language processing, software architecture, software maintenance, traceability

1 INTRODUCTION

Beyond code, software systems are described by architecture documentation and architecture models. Trace links tie these artifacts together, so a change in one can be followed into the others. They support impact analysis, change propagation, and compliance review, and without them architectural knowledge erodes as the system ages [17, 32, 46]. Recovering these links automatically is a long-standing open problem. Recent LLM-based approaches show success, with the strongest reaching 84% F_1 [30]. However, can practitioners rely on these links? On an established benchmark [28], we found that doc-code gold links are highly concentrated across components: the three largest components hold 69.3–99.3% of component-code links. A tool can therefore top the benchmark yet never link a whole component that the documentation describes, and the score fails to present this miss sufficiently. This paper answers the question with both an improved traceability link recovery method and a measure that exposes these misses.

We study this in architecture traceability: linking a documentation sentence to model components (doc-model), and through them to code (doc-code). Prior work follows two lines. The lexical line, such as SWATTR [39], scores each candidate link with linguistic patterns. The LLM line, such as LiSSA [29], classifies each documentation-and-target pair with one LLM call. The strongest approach, Artemis, combines lexical and LLM technique, uses an LLM to extract architecture entities with their aliases, then matches them to the model by name and embedding similarity [30]. In practice, these approaches both miss true links and produce false ones. Components whose documentation never uses their model name receive few or no links, and components whose name overlaps with common words receive links the text does not support. Furthermore, we found that on the benchmark, baselines abandon up to 7.1% of documented components entirely. These approaches share one root cause: they mainly treat word overlap as the signal for a link. SWATTR

2026. XXXX-XXXX/2026/9-ART \$15.00

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

accepts a link when a name scores above a similarity threshold. LiSSA classifies each pair in one LLM call that discards its own chain of thought. Artemis extracts entities and aliases for computing similarity score, then similar to SWATTR, it accepts a link when the score is above a threshold. When a word matches, but the sentence is not about the component, no step catches the false link. When a sentence refers to a component without using its name, the reference is seldom considered.

A trace link between an architecture document and a model component is established when that sentence discusses the architectural role of a component. A sentence can mention a component without being about it, and be about a component without mentioning it. For example, “database access” mentions DB but describes a responsibility of another component, while “It” mentions no component but refers to DB through the preceding sentence (Figure 1). Our insight is that TLR is therefore not about deciding whether a sentence matches a component. It is to find the specific evidence in the document that supports or refutes each candidate link, and to judge the candidate against that evidence.

We present ArchLinker, a training-free LLM workflow that recovers trace links by finding and judging textual evidence. It discovers each document’s component vocabulary, generates candidates through how each reference is written, and judges each candidate against the evidence that produced it. Three parts work together: a shared alias-discovery stage and two recovery routes, each with a candidate generator and a judge. The named-reference route uses the name candidate generator to find complete names, approved aliases, and name words; the coreference route uses the coreference candidate generator to resolve an implicit reference against its antecedent. Each judge is a context-augmented LLM-as-a-judge: a plain judge scores an output alone, but ours receives the evidence the candidate rests on, such as the matched span, how the sentence writes the name, and, for an implicit reference, the antecedent sentence. So, it can reject trace links the evidence does not support. Splitting the reference forms expands candidate coverage without adding a second judging criterion: both written forms are judged under *one* rule, and what separates them is a field of the evidence rather than a second rubric.

Percentage-point (pp) gaps below are rounded from the underlying scores. On the ArDoCo benchmark[28], ArchLinker reaches a doc-model average F_1 of 0.936, 13 pp above the strongest baseline Artemis re-run on the same backend, and 0.955 average F_2 , 13 pp above it [2, 30, 38]. On doc-code it reaches an average F_1 of 0.879 (+7 pp) and an average F_2 of 0.896 (+5 pp).

These gains are measured by standard link-level metrics, but do those metrics tell the full story? Our second contribution starts from a measured property of doc-code links: they are highly concentrated across components. On the benchmark, excluding the largest three architectural components, the remaining components hold between 0.7% and 30.7% of trace links. Within each project, link-level F_1 pools link across components, so components with fewer links have less influence on that score. If a tool misses that component entirely, the score barely moves, but the architect has a gap they will not see. This makes link-level average too coarse to evaluate how recovery varies for the whole architecture model. We therefore introduce an architecture-driven evaluation suite that measures how recovery covers the full architecture model, not just the aggregate link count (Section 4). Its key idea is to make every documented component visible in the score, regardless of how many links it holds. This metric suite also revealed that although Artemis improves over TransArc[30] on average, it scores lower on small components than the lexical approach. In addition, it shows the strength of ArchLinker. On doc-model, ArchLinker covers components that every prior tool drops. On doc-code, ArchLinker leads TransArc by 23 pp on the per-component harmonic mean, against 8 pp at the link level.

This work makes three contributions:

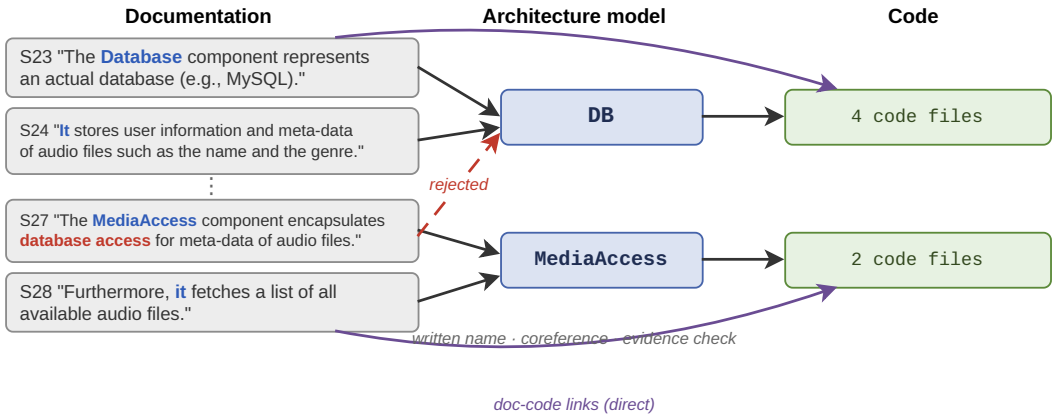


Fig. 1. MediaStore ground-truth trace example. Solid arrows are recorded links; the dashed arrow is a candidate rejected by the recorded judge.

- **ArchLinker**, a training-free, multi-stage LLM workflow for doc-model trace-link recovery (Section 3).
- an **architecture-driven evaluation suite** of three size-aware metrics for architecture trace-link recovery. (Section 4).
- An **empirical study** showing that ArchLinker improves over the strongest baseline by 13 pp F_1 (13 pp F_2) on doc-model and 7 pp F_1 (5 pp F_2) on doc-code, with larger gains on the size-aware measures.

Section 2 motivates the task and the concentration of its doc-code links. Section 3 presents ArchLinker. Section 4 defines the evaluation suite. Section 5.2 and Section 6 give the experimental design and results.

2 BACKGROUND AND MOTIVATION

2.1 Motivating Example

A trace link connects a documentation sentence to an architecture component (doc-model) or to code (doc-code). Figure 1 uses four sentences and their links from the MediaStore benchmark. Sentence 23 writes “The Database component” for what the model calls DB, and Sentence 24 refers back to it only as “It”. Sentence 27 names MediaAccess and contains “database access”, but the gold standard assigns no DB link because the phrase describes a responsibility rather than the component. Sentence 28 then refers back to MediaAccess as “it”. The excerpt therefore shows three problems: a component whose document name differs from its model name, two sentences that refer to components without naming them, and a false candidate that the sentence’s evidence does not support.

The next two subsections develop these problems in detail.

2.2 Challenges of Trace-Link Recovery

Lexical similarity approaches catch a link only when the sentence contains the component’s name. Documentation does not always work that way. After a component is introduced, later sentences can refer to it by aliases and pronouns, not by its full name. This gap between how a document refers and how a name match works causes two problems: it misses true links and accepts false ones.

Recovering these links presents three challenges shown in Figure 1.

(1) The document and the model can use different terms for the same component. The model component is DB, but Sentence 23 writes “The Database component.” A method must discover that “Database” names DB. (2) Different reference forms provide different kinds of evidence. Sentence 23 writes the component name via paraphrasing. Sentence 24 uses “It,” which can only be resolved from the surrounding context: sentence 23 introduces DB. Likewise, Sentence 28 uses “it” for the MediaAccess named in Sentence 27.

(3) Finding a plausible candidate does not establish a trace link.

Even when a sentence does refer to a component, the resolution can be wrong. A referring expression such as “this” may denote an operation or a data object rather than a component. Once “Database” is known as a form of DB, a match on “database access” in Sentence 27 proposes a DB link. But the phrase describes a responsibility of MediaAccess, not the DB component.

A recovery method must therefore (i) learn document specific vocabulary to understand coined terms in context, (ii) analyze references across different types of sentences, and (iii) reduce false-positive links by re-evaluating candidate links.

2.3 Architecture-Driven Evaluation

We conducted an empirical study and found doc-code gold links are highly concentrated across components. The components with the most links account for most link assignments, leaving smaller components with little weight in link-level metrics. The standard metrics like precision, recall, and F-measure averaged over all links pool links across components. Component link counts differ widely, and the resulting concentration is what the standard metric can hide.

On JabRef, directories map to files ranging from a single file to hundreds. Three large components expand into 99.3% of the gold links, while preferences component expands into only 0.44% (Figure 2, left axis). Link-level F_1 is therefore dominated by those three. Yet 20% of the documented sentences describe preferences, 46x its share of links. Similarly, 17.6% of the code’s cross-component dependencies point at it, 40x its link share. A component that a reader meets in one sentence out of five, and that a sixth of the code depends on, is nearly invisible to the score.

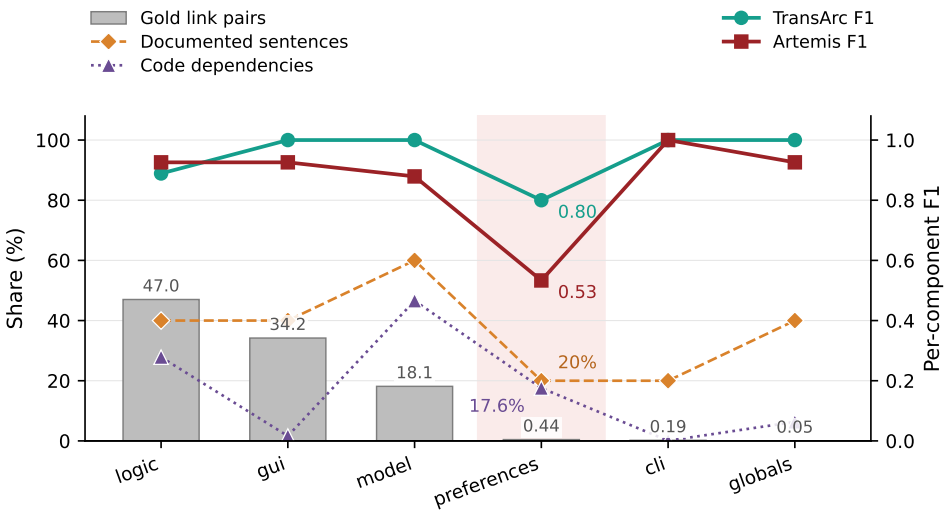


Fig. 2. JabRef component importance and per-component F_1 .

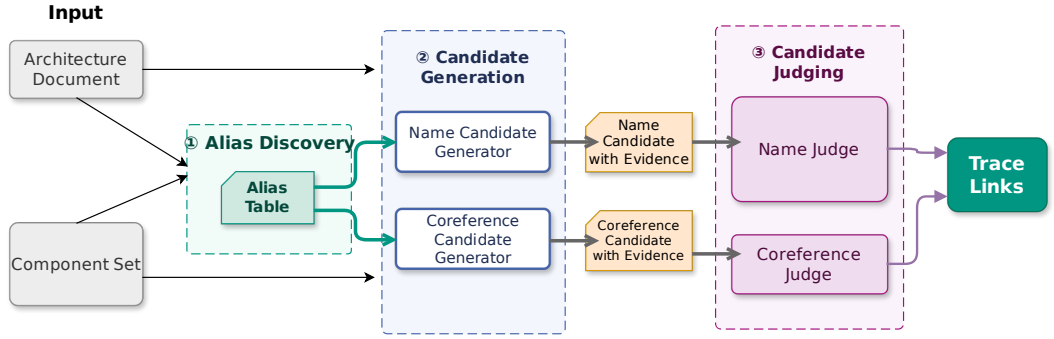


Fig. 3. Workflow overview for ArchLinker. It builds one shared alias table, then generates and judges candidates through separate named-reference and coreference routes.

This concentration also affects comparisons of existing tools. On JabRef, Artemis is weakest on exactly this component: it scores 0.53 on preferences against TransArc’s 0.800, and abandons it entirely in one run of three (Figure 2, right axis). Its F_1 on the three large components stays near 0.9, so the link-level average barely changes because preferences only carries 0.44% of the links. Across the benchmark this reverses the ranking: Artemis leads TransArc by 1.0pp of link-level F_1 while behind it by 4.7pp on the worst component and 4.1pp on the harmonic mean. Link-level F_1 can therefore rank a tool that covers fewer architectural components above one that covers more. This motivates us to quantify doc-code link concentration across all five projects and introduce size-aware metrics that expose dropped components.

3 ARCHLINKER

We propose ArchLinker, an LLM workflow that requires no task-specific model fine-tuning or labeled training data and recovers trace links between an architecture document and an architectural model. Its input is a document D with sentences $S = \{s_1, \dots, s_n\}$ and a component set $C = \{c_1, \dots, c_m\}$. Its output is a link set $L \subseteq S \times C$. Each link $(s_i, c_j) \in L$ asserts that sentence s_i makes an architectural claim about component c_j as a participant in the documented system.

Recovering these links presents three challenges: (1) vocabulary mismatch between document and model; (2) different reference forms require different evidence; (3) finding a plausible candidate does not establish a trace link. A known term may serve another purpose in the sentence, and a referring expression may denote an operation or data object rather than a component.

Design of ArchLinker. Three design decisions address the three challenges. First, ArchLinker builds shared document-specific knowledge to address the vocabulary mismatch between the document and model. Second, ArchLinker separates recovery into the named-reference route and coreference route because different reference forms require separate tailored solutions. Third, every candidate is evaluated with the evidence for that case, forming an evidence-backed judging.

Figure 3 organizes these decisions along two axes. The columns are three functional stages: alias discovery, candidate generation, and candidate judging. The rows after alias discovery are the two end-to-end recovery routes. Stage 1 builds an alias table shared by both routes. In Stage 2, name candidate generator finds candidates supported by text written in the target sentence, while coreference candidate generator resolves references that depend on surrounding sentences. In Stage 3, name judge and coreference judge evaluate the candidates from their respective routes. The workflow associates each candidate with a structured *evidence bundle*—the matched span, how the sentence writes the component name, and for implicit references, the antecedent sentence—and each

Named Links	
(a) Link cases	Approved alias: database -> DB
Case 1 S1 The <u>DB</u> component represents an actual database (e.g., MySQL).	Case 2 S2 The <u>MediaAccess</u> component encapsulates database access for meta-data of audio files.
(b) Judge Input: candidate link & evidence	
Case 1: "S1" → database Span=DB, Evidence: written=alias	Case 2: "database" → database Span=database, Evidence: written=exact
(c) Judge Output:	
claim="The database componen written in alias is discussed architectural responsibility" approve=true, objection=none	claim=none, approve=false objection="database" describes the kind of data access, not the database component.

Fig. 4. Accepted and rejected named-reference candidates.

evidence-backed judge receives that bundle with the candidate. The final output is the set union of accepted links from both routes.

3.1 Alias Discovery

Stage 1 identifies document-specific terms that refer to model components. These terms can include abbreviations, alternate names, and words of multi-word names when the document uses the word for the complete component. For example, one document can write both “BigBlueButton web application” and “the web app” for the model component BBB web. This stage is inspired by prior work on glossary-term extraction [5, 31], software vocabulary mining [26, 35], and architecture-entity extraction in Artemis [30].

An LLM first proposes candidate aliases from the document and component set. An internal alias judge then evaluates whether the document establishes each proposed mapping. It rejects a term when the term is generic vocabulary or identifies something other than one named component. If an approved term maps to multiple components, ArchLinker discards the term because it does not identify a unique component. The remaining mappings form the alias table used during candidate generation and candidate judging. In summary, alias discovery builds one shared table that both recovery routes use during candidate generation and judging.

3.2 Named-Reference Route

The named-reference route handles candidates supported by written evidence in the target sentence. Its name candidate generator combines two deterministic lexical scans. The first finds complete component names and approved aliases. The second, the *name-word scan*, finds individual words of multi-word component names even when the sentence uses a different grammatical form, via NLP lemmatization.

The two scans produce one candidate stream. The generator records the matched span for each remaining candidate, and derives a factual written value from the sentence, component name, and approved aliases. The value is exact when the sentence writes the complete catalog name. It is alias when the sentence writes an approved alias. It is part when the name-word scan finds a name word in the same or a different grammatical form. Or it is qualified name when the name occurs inside a code identifier, such as “storage” in storage.api for the component Storage. Such an occurrence may denote a code path rather than the architectural component itself.

(a) Link cases		Coreference Links	
S3 The <u>Database</u> component represents an actual database (e.g., MySQL).	S5 The <u>ReEncoder</u> component converts the bit rates of audio files.		
S4 <u>It</u> stores user information and meta-data of audio files.	S6 <u>This</u> can result in reduction of file sizes.		
(b) Judge Input: candidate link & evidence			
Case 3: pronoun/role-ref → DB reference="It", antecedent_sentence=S3	Case 4: pronoun/role-ref → Reencoding reference="This", antecedent_sentence=S5		
(c) Judge Output			
claim= "It" refers to Database component and describe its architectural responsibilities, approve=true, objection=none;	claim=none, approve=false objection= "This" refers to the conversion operation, not unambiguously to the component.		

Fig. 5. Accepted and rejected coreference candidates.

The name judge evaluates each candidate together with its evidence bundle. It accepts a candidate when the sentence makes an architectural claim about the proposed component as a participant. It rejects a candidate when the matched expression serves another purpose in the sentence.

Figure 4 illustrates this distinction. In the accepted case, the approved alias denotes the proposed component, so name judge accepts the candidate. In the rejected case, “database” modifies the access responsibility of MediaAccess rather than denoting the proposed component. The same lexical term generates both candidates, but their sentence-level evidence leads to different judgments.

To sum up, the named-reference route recovers links supported by written evidence in the target sentence, and the name judge rejects candidates where the matched expression serves another purpose.

3.3 Coreference Route

The coreference route handles sentences that refer to a component without writing its name. Such a sentence may use a pronoun or noun phrase whose antecedent appears in the surrounding discourse. Because a lexical scan cannot determine what this expression denotes, coreference candidate generator uses an LLM to propose the resolution.

For each proposal, coreference candidate generator records the target sentence, the referring expression, the proposed component, the antecedent sentence, and the antecedent text. These candidates then enter into the judge. First, to ensure the antecedent is reliable for the reference, a pre-hook verifies that the cited antecedent sentence writes either the component’s exact name or via alias. A candidate that fails this condition is rejected before coreference judge is called. For each remaining candidate, coreference judge receives the referring expression, the cited antecedent, the target sentence, and its preceding sentence. It accepts the candidate only when the referring expression unambiguously denotes the proposed component and the target sentence makes an architectural claim about that component.

The two cases in Figure 5 illustrate the coreference judgment. In Sentence 4, “It” refers to the DB component introduced in local Sentence 3, and the target sentence states an architectural responsibility of that component. The coreference judge therefore accepts the candidate. In the rejected case, “This” in local Sentence 6 denotes the conversion operation described in local Sentence 5 rather than unambiguously denoting Reencoding. The surrounding discourse concerns reencoding, but that context alone does not establish the proposed component reference. The coreference judge therefore rejects the candidate.

Table 1. Gold link concentration by task. Sent. counts document sentences; kLOC sums the selected primary-language cloc counts in thousands. Comp. counts all components in the base architecture model. Each Links column counts distinct gold pairs; Med., Max., Gini, and Top-3 share (%) use that task’s gold-reachable components. Shared doc-code files contribute to each mapped component; unmapped files contribute only to Links.

Project	Sent.	kLOC	Comp.	Doc-model					Doc-code				
				Links	Med.	Max.	Gini	Top-3	Links	Med.	Max.	Gini	Top-3
MediaStore (MS)	37	4	14	31	3	7	.31	51.6	59	4	28	.54	74.6
TeaStore (TS)	43	12	11	27	5	6	.18	63.0	707	78	320	.47	86.8
Teammates (TM)	198	145	8	57	5	15	.26	59.6	8,097	861	3,622	.48	77.1
BigBlueButton (BBB)	87	159	12	62	4	14	.37	54.8	1,529	152.5	732	.53	69.3
JabRef (JR)	13	157	6	18	4	6	.22	77.8	8,268	768	3,888	.59	99.3

Altogether, the coreference route recovers links from implicit references, under a structural constraint that the antecedent sentence writes the component’s name.

4 AN ARCHITECTURE-DRIVEN METRIC SUITE

Doc-code gold links are highly concentrated across components, so link-level averaged metrics can hide whether a system covers every documented component. This section quantifies that concentration across all five projects and builds a metric suite that exposes the components a system leaves untraced.

4.1 Link Concentration in the Benchmark

Breaking down by components, document to code trace links have a highly unequal distribution. Table 1 compares gold links per component at both task grains using the Gini coefficient (0 = uniform, and larger values indicate greater concentration). Doc-code Gini ranges from 0.47 on TeaStore to 0.59 on JabRef; doc-model ranges from 0.18 on TeaStore to 0.37 on BigBlueButton. Doc-code has the higher per-component Gini in each of the five projects. To make this concrete: a Gini of 0.59 means that if all trace links were income, the richest component would earn roughly what the bottom 80% components earn combined. The median and maximum link counts also show the concentration: on Teammates the median component has 861 trace links while the largest has 3,622; on JabRef the ratio is 768 to 3,888. Overall, the three largest components hold between 69.3% (BigBlueButton) and 99.3% (JabRef) of the doc-code per-component link assignments. Link-level metrics mainly reflect these few large components, hiding whether the system recovers the rest. An architect’s primary job is not reading the benchmark average score. They read the architecture document with models, and a hole in it is relevant regardless of the benchmark score. We therefore define a suite that gives components explicit visibility and reports what the link-level average buries.

4.2 The Metric Suite

Our suite has three design goals: adjust the weight of every component, stay complementary beside link-level F_β so a reader can see what the standard metric hides, and require no additional manual annotation beyond the benchmark. It adapts established evaluation principles of concentration measurement to the architecture trace-link recovery setting. The contribution is on the application side: connecting the unequal link distribution measured above to a concrete evaluation gap that none of the existing TLR benchmarks address. (i) worst-component F_β reports the component a system recovers worst on the doc-code task; (ii) harmonic-mean F_β aggregates per-component

performance while emphasizing low-scoring components; and (iii) the component miss rate reports the weighted fraction of components a system never recovers on the doc-model task. For the doc-code metrics, we define the family over the F_β score,

$$F_\beta = \frac{(1 + \beta^2) P R}{\beta^2 P + R}, \quad (1)$$

where F_1 ($\beta = 1$) weights precision and recall equally, and F_2 ($\beta = 2$) weights recall more. Previous studies have suggested that both are essential in the trace-link recovery task and have recommended reporting F_2 when recall is more important than precision [1, 21].

Doc-code: worst-component and harmonic-mean F_β . On the doc-code task, the concentration of component-code files lets a system score high on link-level F_β while missing a whole component. Therefore, we propose two finer-grained metrics at the component level. Let $\mathcal{K}$ be the components in the architecture model with at least one trace link; Each component k is linked to a set of code files; $F_\beta(k)$ denotes F_β computed over only the links whose target belongs to component k . The worst-component F_β is the minimum over the components:

$$F_\beta^{\min} = \min_{k \in \mathcal{K}} F_\beta(k). \quad (2)$$

A single missed component drives it to zero, even when link-level F_β stays high. Like F_β , Worst-component F_β ranges in $[0, 1]$ and is a floor. It reports the score of a system's weakest component and says nothing about how many components sit near it. A missed component is invisible to link-level average F_β in proportion to how small it is, so a small component contributes relatively few outcomes to the aggregate. The minimum changes that weighting. Because minimum is determined by a single component and potentially noisy, we also report the harmonic mean of the per-component F_β :

$$F_\beta^H = \frac{|\mathcal{K}|}{\sum_{k \in \mathcal{K}} 1/F_\beta(k)}. \quad (3)$$

It is built on the insight that a low score should be expensive to average away. It gives low component scores greater influence on the aggregated result. When any component scores $F_\beta(k) = 0$, the harmonic mean is undefined (division by zero); we define $F_\beta^H = 0$ in that case, so a system that abandons a component cannot recover the harmonic mean by scoring well everywhere else. For example, suppose five components have (20, 40, 1500, 2800, 3900) links, reflecting the concentration in Table 1, and their per component $F_\beta(k)$ scores are (0.2, 0.9, 0.9, 0.9, 0.9). On component level, the harmonic mean by component is about 0.53, versus about 0.8983 for the link-count-weighted arithmetic mean of these component scores. With harmonic mean, F_β of one component at 0.2 pulls the mean further than four components at 0.9 can restore it.

Doc-model: component miss rate. Doc-model per-component link counts have lower Gini than doc-code counts (Table 1), so their links are less concentrated. But a system can still miss a component entirely, leaving its documented sentences with no correct link. Let R_k denote recall on component k and $\mathcal{S}_k$ for the linked sentences. A component is *abandoned* when $R_k = 0$, that is, when the system recovers none of its correct links.

A component the system abandons still has documentation describing it, and CMR is built on this idea: it weights the miss by the sentences describing the component.

$$\text{CMR} = \frac{\sum_{k: R_k=0} |\mathcal{S}_k|}{\sum_{k \in \mathcal{K}} |\mathcal{S}_k|} \in [0, 1], \quad (4)$$

CMR = 0 means every documented component is linked at least once, so no component's documentation is silently lost.

On doc-code the suite reports worst-component F_β and harmonic-mean F_β , each at both $\beta = 1$ and $\beta = 2$. On doc-model it reports the component miss rate. Section 5.2.2 applies this suite to all systems on the benchmark.

5 EXPERIMENTAL EVALUATION

We study four research questions:

5.1 Research Questions

RQ1: *Does ArchLinker recover trace links better than existing approaches?*

Motivation: Existing approaches either match component names lexically [39] or recover combining a single LLM call with similarity matching [30]. ArchLinker instead separates candidate generation by reference form and uses an evidence-backed judge to check the generated candidates. We evaluate whether this workflow improves recovery on both doc-model recovery task and the downstream doc-code task.

RQ2: *What component-level behavior do architecture-driven metrics tell beyond standard link-level metrics?*

Motivation: Link-level precision, recall, and F_β weight every link equally. We found that doc-code links are highly concentrated across components, so components with many links can dominate these aggregate scores. The link-level average is too coarse to show how recovery varies across components. We therefore examine whether proposed size-aware metrics reveal component-level misses which is difficult to see in the standard metrics.

RQ3: *How much do the judges inside ArchLinker contribute to the final result?*

Motivation: One key feature of ArchLinker is that each recovery route pairs candidate generation with a corresponding evidence-backed judge. The judges can remove such false positives, but they may also reject true links. We therefore measure which true and false links each judge rejects and how disabling the judges changes the final precision-recall trade-off.

RQ4: *What does each module of ArchLinker contribute on its own?*

Motivation: ArchLinker recovers links through two reference forms: named mentions and coreference mentions. The two recovery routes share the alias table constructed before candidate generation. We examine whether the reference forms recover complementary links and measure how recovery changes when the shared knowledge is unavailable.

5.2 Experiment Design

To answer the four research questions, we run four families of experiments on the ArDoCo benchmark, the largest benchmark to the best of our knowledge [28]. It contains five Java open-source systems with hand-curated architecture docs, architecture models, and gold standard trace-links at two grains, document-to-component and document-to-code. We use the released configurations of the deterministic baselines. Because ArchLinker and Artemis are LLM-based therefore non-deterministic, we run each three times and report the mean. The deterministic baselines are run once. To avoid depending on one model, we run ArchLinker on two language-model backends, GPT-5.6-terra and GPT-5.6-luna. Prompts, configurations, raw model responses, and experiment outputs are included in the replication package [2].

5.2.1 Experiments for RQ1. We evaluate recovery at two grains.

- (1) *Doc-model.* For every sentence in the architecture document, a system recovers links between the architecture model components described by that sentence. We compare recovered links with the doc-model gold standard. We compare ArchLinker with two baselines: SWATTR [39],

a prior lexical string-similarity pipeline; and Artemis [30], the strongest directly comparable LLM-based baseline. This is the task where ArchLinker can directly show its strength.

- (2) *Doc-code*. Following prior evaluations [30, 38], we compose recovered doc-component links with component-to-file links to obtain sentence-to-file links. The component-to-file links are recovered from the same approach ArCoTL [38].

We compare ArchLinker with two baselines: TransArc [38], the strongest lexical-based approach; and Artemis composed through the ArCoTL links, as an LLM baseline at the doc-code grain. We do not compare with a simple few shot baseline LiSSA [29] because it has been shown less effective than Artemis and costs significantly more.

For both tasks, we report precision, recall, F_1 , and F_2 per project and macro-averaged across the five projects. We also report input and output tokens for each project. We average token counts across runs for each project. RQ1 uses the standard metrics the baselines were originally evaluated by.

5.2.2 Experiments for RQ2. The previous RQ evaluates systems with the standard link-level metrics. RQ2 focuses on how recovery varies across components within those projects. We evaluate the same recovered links with the size-aware suite defined in Section 4.2. The recovered links remain fixed, and only the view used to evaluate them changes.

On doc-model, the suite adds the component miss rate (CMR, Equation 4), which measures the weighted rate of components where a system recovers no correct link. Missed components are weighted by the number of sentences describing it.

On doc-code, we add worst-component and harmonic-mean per-component F_1 and F_2 . These metrics give each component explicit visibility rather than weighting components by their number of links.

5.2.3 Experiments for RQ3. We build three ablations of ArchLinker that remove judges: one without the name judge, one without the coreference judge, and one without either judge.

- (1) *NoName*: accept every candidate retained by the deterministic checks in the named-reference route without applying the name judge.
- (2) *NoCitation*: accept every candidate retained by the deterministic checks in the coreference route without applying the coreference judge.
- (3) *NoValidator*: accept candidates retained by the deterministic checks in both routes without either judge.

For each judge, we count the true and false links it keeps and rejects. We also compare the resulting F_1 and F_2 with the full approach to quantify the judge's net effect on the final results.

5.2.4 Experiments for RQ4. We analyze the contribution of the recovery routes and the shared alias knowledge in two complementary ways. First, we measure the overlap between the two recovery routes: the named-reference route and the coreference route. For each route, we count the true links recovered uniquely by that route and evaluate the links produced when that form is used on its own. This measures each route's scope and whether the two routes recover complementary parts of the gold standard.

Second, we perform an ablation of the shared knowledge module. We disable alias discovery, re-run the workflow, and compare the resulting links with the full workflow. Both routes remain unchanged, isolating the contribution of the shared alias knowledge.

6 RESULTS

The quality tables report arm s126 on the five ARDoCo projects with GPT-5.6-terra; detailed results and the GPT-5.6-luna mirror are in Appendix A. For the stochastic systems ArchLinker and

Table 2. RQ1 link scores by project on GPT-5.6-terra. Project abbreviations: MS=MediaStore, TS=TeaStore, TM=Teammates, BBB=BigBlueButton, JR=JabRef.

Project	Task	ArchLinker Precision/Recall; F ₁ /F ₂	Artemis Precision/Recall; F ₁ /F ₂	SWATTR / TransArc Precision/Recall; F ₁ /F ₂
MS	doc-model	.91±.00/ 1.00 ±.00; .95/.98	.90±.06/.97±.00; .93/.95	.94 /.55; .69/.60
	doc-code	.85±.00/.88±.00; .87/.88	.92±.05/ .93 ±.00; .93/.93	.96 /.42; .59/.48
TS	doc-model	1.00 ±.00/ 1.00 ±.00; 1.00/1.00	.57±.01/.98±.02; .72/.85	1.00 /.74; .85/.78
	doc-code	1.00 ±.00/ 1.00 ±.00; 1.00/1.00	.58±.00/.98±.02; .73/.86	1.00 /.71; .83/.75
TM	doc-model	.81±.01/ .94 ±.01; .87/.91	.92 ±.07/.64±.06; .76/.68	.60/.86; .71/.79
	doc-code	.80±.00/.74±.00; .77/.75	.93 ±.09/.61±.06; .73/.65	.75/ .90 ; .82/.87
BBB	doc-model	.87±.00/ .90 ±.01; .89/.89	.80±.08/.70±.06; .75/.72	.90 /.71; .79/.74
	doc-code	.68±.00/ .93 ±.01; .78/ .86	.70±.12/.85±.02; .76/.81	.82 /.84; .83 /.84
JR	doc-model	.95 ±.05/ 1.00 ±.00; .97/.99	.83±.06/.96±.06; .89/.93	.90/ 1.00 ; .95/.98
	doc-code	.95 ±.07/ 1.00 ±.00; .97/.99	.85±.10/ 1.00 ±.00; .92/.96	.89/ 1.00 ; .94/.98
Avg	doc-model	.91 ±.01/ .97 ±.00; .94/.95	.81±.02/.85±.02; .81/.83	.87/.77; .80/.78
Avg	doc-code	.86±.01/ .91 ±.00; .88/.90	.80±.01/.87±.01; .81/.84	.89 /.78; .80/.78

Precision and recall include sample SD across three runs, rounded to two decimals on the score scale; Average SD uses the three per-run project means. SWATTR supplies deterministic doc-model results and TransArc deterministic doc-code results.

Artemis, scores are means of three runs. The inference-usage table separately identifies its models and run sets. Percentage-point gaps below are rounded from the underlying scores.

6.1 Answer to RQ1

Table 2 reports precision, recall, F₁, and F₂ for every system per project on both tasks, doc-model and doc-code. The gain is smallest on MediaStore, where the doc-model F₁ differs by +2pp and F₂ by +3pp. On the other four projects the F₁ lead ranges from +8pp (JabRef) to +28pp (TeaStore). On average, ArchLinker lifts the average F₁ from Artemis’s 0.81 to 0.94, a gain of +13pp after rounding; the lexical SWATTR pipeline reaches 0.80. The recall-weighted average F₂ moves by a similar margin: 0.83 → 0.95 (+13pp).

The gain is through both precision and recall improvement. On doc-model ArchLinker leads on both metrics of the score, precision 0.91 against 0.81 (+10pp) and recall 0.97 against 0.85 (+12pp); on doc-code it leads by +6pp precision and +4pp recall.

The same picture holds on the downstream doc-code task, where ArchLinker reaches code files transitively. ArchLinker lifts the doc-code average F₁ from Artemis’s 0.81 to 0.88 (+7pp) and the average F₂ from 0.84 to 0.90 (+5pp), where the heuristic TransArc pipeline reaches 0.80 and 0.78. These are observed downstream scores; composition need not preserve the doc-model ranking.

The direction holds on four of the five projects over Artemis and three over TransArc. On MediaStore over Artemis, a +2pp doc-model F₁ gain becomes a −6pp doc-code F₁ decrease. A doc-model advantage does not necessarily persist after transitive composition. This is because ArchLinker’s recall is focused on small components which transitively link to fewer doc-code links.

Table 3 reports mean input and output tokens across three runs per project. Across five projects, ArchLinker used 98.6 thousand input tokens. Its output total was 22.7 thousand tokens. Artemis used 20.0 thousand input tokens and 23.4 thousand output tokens. These two approaches use similar amounts of output tokens, while ArchLinker consumes 4.9× more input tokens because of the multi-stage judge workflow. At the GPT-5.6-terra pricing used here (US\$2/12 per million input/output tokens, as of September 2026), the five-project total costs approximately US\$0.47 for ArchLinker and US\$0.32 for Artemis.

Table 3. Averaged cost per project, with estimated cost at list pricing as of September 2026.

System	Input/Output (k tokens)						Cost (US\$)
	MS	TS	TM	BBB	JR	Total	
ArchLinker	9.9/2.1	11.4/3.5	43.8/7.7	27.4/7.3	6.0/2.1	98.6/22.7	0.47
Artemis	2.6/2.8	2.9/3.5	7.7/8.6	4.8/6.1	2.0/2.5	20.0/23.4	0.32

Table 4. RQ2 size-aware metrics by project on GPT-5.6-terra.

Approach	Proj.	doc-model				doc-code						Proj.	doc-model				doc-code					
		Link	CMR			Link	Worst	Harm.					Link	CMR			Link	Worst	Harm.			
		F ₁	F ₂	%		F ₁	F ₂	F ₁	F ₂	F ₁	F ₂		F ₁	F ₂	%		F ₁	F ₂	F ₁	F ₂	F ₁	F ₂
AL	MS	.95	.98	0.0		.87	.88	.72	.74	.96	.96	BBB	.89	.89	0.0		.78	.86	.44	.56	.81	.83
AT		.93	.95	0.0		.93	.93	.75	.85	.96	.97		.75	.72	1.1		.76	.81	.22	.19	.45	.45
S/T		.69	.60	35.5		.59	.48	.00	.00	.00	.00		.79	.74	0.0		.83	.84	.57	.45	.84	.80
AL	TS	1.00	1.00	0.0		1.00	1.00	1.00	1.00	1.00	1.00	JR	.97	.99	0.0		.97	.99	.94	.97	.98	.99
AT		.72	.85	0.0		.73	.86	.43	.65	.67	.82		.89	.93	3.7		.92	.96	.53	.61	.61	.64
S/T		.85	.78	0.0		.83	.75	.67	.56	.84	.77		.95	.98	0.0		.94	.98	.80	.91	.94	.98
AL	TM	.87	.91	0.0		.77	.75	.72	.61	.80	.78	Avg	.94	.95	0.0		.88	.90	.76	.78	.91	.91
AT		.76	.68	11.7		.73	.65	.41	.33	.50	.44		.81	.83	3.3		.81	.84	.47	.52	.63	.66
S/T		.71	.79	0.0		.82	.87	.54	.65	.76	.82		.80	.78	7.1		.80	.78	.51	.51	.68	.67

AL = ArchLinker; AT = Artemis; S/T = SWATTR for doc-model and TransArc for doc-code.

Answer to RQ1. ArchLinker leads Artemis by 13pp doc-model and 7pp doc-code average F₁ after rounding. Corresponding gaps to SWATTR and TransArc are 14pp and 8pp.

6.2 Answer to RQ2

Table 4 reports the size-aware suite for ArchLinker, TransArc, and Artemis on both tasks. On doc-model, the component miss rate separates the systems: 0.0% for ArchLinker against 3.3% and 7.1%. This means ArchLinker covers more components than previous approaches: it reaches 0% CMR on all five projects. Artemis loses in three projects (up to 11.7% on Teammates) and SWATTR in one (35.5% on MediaStore). A component the CMR notices is one a developer would notice as entirely missing from the traceability matrix.

On doc-code, the standard link-level F₁ shows the *smaller* gap: ArchLinker leads Artemis by +7pp after rounding, while each size-aware measure gives components a different weighting. The size-aware metrics open it. ArchLinker reaches a worst-component F₁ of 0.76 against Artemis's 0.47 (+30pp), and a harmonic per-component F₁ of 0.91 against 0.63 (+28pp). Against the deterministic TransArc pipeline the same pattern holds with a smaller spread: +8pp link-level, +25pp worst-component, +23pp harmonic.

The size-aware metrics also reveal room for improvement that the standard average hides. Even ArchLinker, despite a doc-code average F₁ of 0.88, reaches a worst-component F₁ of only 0.44 on BigBlueButton and 0.72 on Teammates. All three systems share this pattern: high link-level averages coexist with weak component-level scores, and the per-component scores expose these differences.

The rankings can also disagree: on Teammates, TransArc exceeds ArchLinker on file F₁ (0.82 versus 0.77), but trails on worst-component F₁ (0.54 versus 0.72). At the macro level, Artemis

Table 5. RQ3 judging configurations

Judges	rejects		keeps		doc-model		doc-code		
	FP	TP	TP	FP	Prec./Rec.; F ₁ /F ₂	CMR%	Prec./Rec.; F ₁ /F ₂	worst F ₁ /F ₂	harm. F ₁ /F ₂
Full	144.3	9.7	185.3	25.0	.91/.97; .94/.95	0.0	.86/.91; .88/.90	.76/.78	.91/.91
name judge	99.0	9.7	170.3	23.0	.75/ 1.00 ; .84/.92	0.0	.76/ .96 ; .84/ .90	.56/.70	.82/.90
coreference judge	53.0	1.3	34.3	2.0	.76/.97; .85/.92	0.0	.77/.92; .83/.88	.66/.76	.85/.89
No judge	0.0	0.0	195.0	166.0	.67/ 1.00 ; .79/.89	0.0	.71/ .96 ; .81/.89	.53/.69	.78/.88

exceeds TransArc on file F₁ (0.81 versus 0.80), but trails on worst-component F₁ (0.47 versus 0.51). These observed reversals show that the measures need not rank systems identically.

Answer to RQ2. On doc-model, ArchLinker reaches a 0.0% component miss rate, versus Artemis's 3.3% and SWATTR's 7.1%. On doc-code the standard F₁ gap is +7pp; the size-aware suite opens it to +30pp worst-component and +28pp harmonic-mean F₁, exposing weak component coverage in the baselines.

6.3 Answer to RQ3

Table 5 reports rejected and kept links alongside four judging configurations. Each row reports for each judge, the number of links it rejects and keeps. It further shows its impact on final results if they are disabled.

The name judge rejects 99.0 false positives while costing 9.7 true trace links; the coreference judge rejects 53.0 against 1.3. Together the two judges reject 144.3 distinct false positives and lose 9.7 true links. Their link counts overlap and therefore do not sum. This is because each judge reads a different kind of *evidence* about many overlapping candidates: the name judge reads the evidence supporting named links, and the coreference judge reads evidence for coreference resolution. These aggregate counts do not identify disagreements on individual candidates.

The judges make the generated link candidates more precise. Switching both off drops link-level F₁ from 0.94 to 0.79, a loss of 15pp, but link-level F₂ only from 0.95 to 0.89, a loss of 6pp. Removing them one at a time costs 10 and 8pp of F₁ (name judge, coreference judge) against 3 and 3pp of F₂ after rounding. This is because a judge only removes links: it is supposed to improve precision only, so a metric that weights recall four times as heavily as precision sees a smaller effect.

The judge-off block shows that trade directly: with both judges off, doc-model precision falls by 24pp (0.91 → 0.67) while recall rises by only 3pp (0.97 → 1.00), so candidate generation is already near the recall ceiling and the judges are buying the precision that is missing. On the size-aware side, judging does not cause any additional component miss: the component miss rate is 0.0% with and without judging.

Therefore the judges turn high-recall candidates into a more precise result.

Answer to RQ3. The two judges together reject 144.3 distinct false positives while losing 9.7 true links; removing both costs 15pp link-level F₁ but only 6pp link-level F₂ after rounding. Thus, judging primarily exchanges a small amount of recall for substantially higher precision.

6.4 Answer to RQ4

ArchLinker links through two recovery routes over one shared alias table, so it is ablated by route and by shared knowledge: named-reference route, coreference route, and the alias table. Table 6

Table 6. RQ4 module ablation

Variant	doc-model		doc-code		
	Prec./Rec.; F ₁ /F ₂	CMR%	Prec./Rec.; F ₁ /F ₂	worst F ₁ /F ₂	harm. F ₁ /F ₂
Full	.91/.97; .94/.95	0.0	.86/.91; .88/.90	.76/.78	.91/.91
name linker only	.91/.88; .90/.89	0.0	.85/.83; .83/.83	.69/.66	.86/.84
coreference linker only	.89/.17; .28/.20	36.3	.89/.17; .27/.20	.01/.01	.03/.02
No knowledge	.95/.81; .87/.83	10.8	.92/.71; .77/.73	.44/.41	.52/.51

reports the four variants on both tasks, each on the reference metrics and on that task’s size-aware metric.

named-reference route alone reaches a doc-model average F₁ of 0.90 and average F₂ of 0.89, already above SWATTR’s 0.80: the named route for written names beats the lexical baseline on its own. Adding coreference route raises them to 0.94 and 0.95, and what it adds is recall: doc-model recall rises from 0.88 to 0.97 while precision is 0.91 in both configurations after rounding. Its 7pp F₂ gain is about 1.6 times its 4pp F₁ gain. named-reference route contributes 151 true positives no other route reaches and coreference route another 15. The component-level scores move with it. With named-reference route alone the worst-component F₁ falls from 0.76 to 0.69 and the harmonic per-component F₁ from 0.91 to 0.86, against 5pp of doc-code file F₁ after rounding. When scored alone against the full gold standard, coreference route scores far lower, 0.28 average F₁ at a 36.3% component miss rate. That is because its scope: it is supposed to fire on coreference links, and they are rare.

The no-knowledge variant disables alias discovery while retaining both linkers. The separately run no-knowledge configuration is 7pp lower in average F₁ (0.94 → 0.87) and 12pp average F₂ (0.95 → 0.83) on doc-model, in these recorded run sets. The F₂ penalty is disproportionately larger because without the alias table, recall falls from 0.97 to 0.81 as components whose document names differ from their model names lose all links; F₂ weights recall four times as heavily as precision, so this recall drop hits F₂ harder than F₁. The component-level differences are larger: worst-component F₁ falls from 0.76 to 0.44 (−32pp) and the harmonic per-component F₁ from 0.91 to 0.52 (−39pp), against −11pp of doc-code file F₁, and the component miss rate rises from 0.0% to 10.8%. The effect is concentrated rather than spread: the three components whose doc-code F₁ falls to 0 without the table, MediaStore’s DB and “Reencoding”, and TeaStore’s ImageProvider, account for 47.5%, 1.7% and 45.3% of their project’s gold doc-code links, respectively. Therefore the modules are complementary rather than interchangeable, and the knowledge module raises component-level scores that the standard metric shows little of.

Answer to RQ4. named-reference route alone beats SWATTR; coreference route lifts average F₁ from 0.90 to 0.94 and average F₂ from 0.89 to 0.95. The separate no-knowledge comparison differs by 7pp average F₁ and 12pp average F₂ after rounding. The full configuration exceeds the no-knowledge configuration by 32pp of worst-component F₁.

6.5 Summary

Together, RQ1–RQ4 show that the workflow, the judges, and the shared alias table each contribute to the final result, and that the size-aware suite exposes component-level differences the standard metric hides. We discuss the threats to validity in [Section 7.3](#).

7 DISCUSSION

7.1 Implications

For researchers. Architectural traceability research should account for uneven link distributions across components, documents, artifact types, and other relevant groupings. As tracing extends across abstraction levels and into larger or more fine-grained artifact spaces, aggregate results can be dominated by densely represented regions and conceal failures in sparsely represented groups. Our framework supports studying this behavior by evaluating recovery at multiple scopes and inspecting candidate generation, contextual judgment, and downstream mapping separately. Researchers can therefore locate where performance changes as distributions become more uneven or the tracing scope broadens, and report results for relevant subsets alongside aggregate performance.

For practitioners. ArchLinker lets practitioners configure the precision–recall trade-off by enabling or disabling the judges. Disabling both judges raises recall by 3 pp but lowers precision by 24 pp (RQ3); practitioners whose cost of missing a link is high can run the unjudged variant and review the extra candidates. On the five benchmark projects, the total inference cost is approximately US\$0.47 at current GPT-5.6-terra pricing, comparable to Artemis’s US\$0.32 (Table 3).

The size-aware metrics require no extra annotation beyond the existing gold standard and can serve as a diagnostic: they show which components a tool under-serves, directing human review where it is most needed.

7.2 Remaining Weaknesses

ArchLinker does not solve the whole task. Its macro-averaged worst-component F_1 on doc-code is 0.76, and on BigBlueButton it falls to 0.44. BigBlueButton has the highest doc-model Gini coefficient among the five projects (0.37, Table 1), meaning its doc-model links are the most unevenly distributed; its 12 components and 87 documentation sentences make uniform coverage difficult. On Teammates, the worst-component F_1 is 0.72, well below the link-level F_1 of 0.77. The size-aware suite makes this headroom visible where the link-level average would suggest near-saturation.

The MediaStore doc-code result illustrates a limitation of transitive trace link: ArchLinker gains 2 pp of doc-model F_1 over Artemis but loses 6 pp on doc-code F_1 . A doc-model advantage concentrated on small components does not survive transitive composition when those components expand into fewer code files.

7.3 Threats to Validity

Internal validity. The LLM systems are stochastic. So we report scores across three runs per project. All five benchmark projects are public open-source systems whose source code and documentation are available on GitHub. The LLM backend has very likely encountered them during pre-training. We therefore re-ran baseline approach Artemis on the same LLM backend ArchLinker uses. Therefore, two systems then share the same exposure and use the same model, so what separates them is the workflow. Secondly, the improvement could come from the extra LLM calls rather than the workflow itself. The ablations (RQ3, RQ4) rule this out by attributing it to specific modules, each contributing a non-redundant slice of links. Alias discovery and coreference resolution may benefit if the model has internalized project-specific vocabulary. A definitive memorization test would require projects the model has not seen, which is difficult to guarantee for any public system.

External validity. We measure on the five Java systems of the ARDoCo benchmark, the largest curated one we know of [28]. Results may not transfer to other languages, industrial systems, or

differently written documents. We mitigate this by using all five projects, which span two orders of magnitude in size and several domains, and by keeping ArchLinker free of any benchmark-specific tuning.

Construct validity. RQ2 assumes that recovering every component matters to developers; our size-aware metrics express this through worst-component and harmonic-mean per-component F_1 on doc-code, and the component miss rate (CMR) on doc-model, weighted missed component rates. These are proxies, so we keep the standard link-level F_1 as the reference and report the size-aware view beside it.

8 RELATED WORK

ArchLinker sits at the intersection of four areas: traceability link recovery, NLP for requirements engineering, LLM-based software engineering, and hallucination control in LLM workflows.

Traceability link recovery. Automated traceability link recovery grew out of the requirements-traceability problem [32, 59], and it is still a long-standing open problem for the community [4]. Lago, Muccini, and van Vliet showed that traces can be maintained to the activities stakeholders must carry out [40]. For two decades, the main family of trace link recovery methods scored each artifact pair by textual similarity from information retrieval methods [9, 19]. Examples include vector-space and probabilistic models [3], latent-semantic indexing [48], artifact-management variants [18], and test-to-code recovery [57, 58]. On the practical side, previous work estimated how many links are still missing [25], showed that the links pay off during maintenance [46], and identified the factors behind successful traceability maintenance [49]. manage traceability information models in practice [50]. Links embedded in commit messages also evolve and decay over time [69]. The learning-based ones include fused textual and structural signals [52], hierarchical Bayesian networks [53], multi-layered bug-to-fix recovery [55], and pre-trained BERT encoders [42]. On the architecture side, the documentation, model, and code must stay consistent [13, 15, 41], and an architecture-centric lifecycle frames how requirements reach code [10]. Design-decision rationale can be codified with tool support [12], and the community has examined how to sustain this knowledge over time [14]. Architecture-based testing checks the code against the description [54]. The ARDoCo line sets the benchmark and the doc-model-code pipeline we compare against [38, 39]. Artemis extends it with LLM-based entity extraction: it builds an entity list from the document and matches entities to the model by name and embedding similarity [30]. Although Artemis extracts knowledge, its link decision is a similarity threshold, not a judgment against the sentence's evidence. So a plausible match that the sentence does not support still becomes a link. LiSSA classifies each documentation-and-target pair with one LLM call augmented by retrieval [29]. The classification is also unverified. ArchLinker differs from both by pairing each candidate generator with a judge that receives the evidence the candidate rests on.

NLP4RE. The recurring obstacle for trace-link recovery is the vocabulary gap: a single concept can appear under many surface forms. The requirements-engineering community addresses this with NLP techniques. Examples include glossary-term extraction [5], NLP-based change-impact analysis over natural-language requirements [6], and practical guidelines for selecting NLP techniques in requirements engineering [60]. Recent work applies LLMs to improve requirements completeness [44, 45]. Mahmoud and Niu found that only focused semantic relationships help close the vocabulary gap for IR-based tracing [47], and text-retrieval work rewrites queries to bridge the same kind of mismatch [33]. Island parsing extracts structured fragments from the same noisy artifacts [7]. Polysemy in identifier and requirement terms further hinders automated tracing [66],

and domain-specific corpora improve how ambiguity is handled [24]. AI-based question answering has been applied to analyze natural-language requirements [23], and collaborative traceability management studies show the organizational factors that determine whether traces survive in practice [68]. Prior work also treats implicit references as a defect to flag [22].

LLM4SE. LLMs now drive many software-engineering tasks [8, 27, 34]. Practitioners have begun to adopt these models in practice [65, 70], including for code review [64]. The closest task is code search, which aims to match natural-language text like requirement or documentation to the code that implements it [51]. Neural models add code summarization [36] and learned semantic search [11, 61]. Spec2Code maps protocol specifications to function-level code using LLM-guided search [67]. Etezadi et al. showed that LLM prompting can match fine-tuned classifiers on legal requirements traceability [21]. Surveys show the move from IR-based to neural code search [43], and retrieval-augmented search now calls LLMs directly [16]. Outdated code-element references in repository documentation compound the evaluation challenge [62]. Treude and Storey note that generative AI raises new evaluation challenges for this entire line of work [63].

Hallucination control in LLM workflows. The main risk of LLM workflows is hallucination. Prior work addresses this through grounding and detection. Retrieval grounding supplies supporting documents before the call [20, 37], generate-then-validate predicts types and checks them post hoc [56], and metamorphic detection compares outputs under input perturbations [71]. Similar to these approaches, judges of ArchLinker reject a hallucinated link using the specific evidence the link rests on.

Position of ArchLinker. ArchLinker is the first architecture-to-code traceability link recovery approach that (i) separates a reusable knowledge layer from the linker, (ii) uses implicit references as link evidence under a structural earlier-sentence constraint, and (iii) matches how hard each LLM judge checks a link to how the link was matched. We compare against SWATTR [39], the lexical-pattern line; TransArc [38], the ARDoCo doc-code pipeline; and Artemis [30], the single-pass-LLM line. [67] [21]

9 CONCLUSION

This paper presents ArchLinker, to our knowledge the first multi-stage LLM workflow for architecture document-to-model traceability link recovery that requires no task-specific fine-tuning. ArchLinker derives an alias table once, then reuses it in two recovery routes—a named-reference route for written names and name words, and a coreference route for implicit references. Each route pairs a candidate generator with an evidence-backed judge that rejects links the evidence does not support. The alias table is critical: without it, worst-component F_1 falls by 32 pp and three components lose all recovery. Alongside the approach we introduce an architecture-driven evaluation suite that adds per-component measures to the standard link-level metric on doc-code, and the component miss rate (CMR) on doc-model. On the ARDoCo benchmark, ArchLinker reaches a doc-model average F_1 of 0.936, 13 pp above the strongest baseline run on the same backend, and carries this gain through to a doc-code average F_1 of 0.879 (+7 pp). Under the size-aware suite this lead widens to +30 pp worst-component F_1 and +28 pp harmonic-mean per-component F_1 . ArchLinker’s macro-averaged worst-component F_1 is 0.76, well below its link-level F_1 , so the suite also keeps this headroom visible where the standard metric alone would not.

DATA AVAILABILITY STATEMENT

Code, data, and results are in the replication package [2].

REFERENCES

- [1] Orlando Amaral, Sallam Abualhaja, Damiano Torre, Mehrdad Sabetzadeh, and Lionel C. Briand. 2022. AI-Enabled Automation for Completeness Checking of Privacy Policies. *IEEE Transactions on Software Engineering* 48, 11 (Nov. 2022), 4647–4674. <https://doi.org/10.1109/TSE.2021.3124332>
- [2] Anonymous. 2025. Replication Package: TODOTODO. todo todo.
- [3] Giuliano Antoniol, Gerardo Canfora, Gerardo Casazza, Andrea De Lucia, and Ettore Merlo. 2002. Recovering Traceability Links between Code and Documentation. *IEEE Transactions on Software Engineering* 28, 10 (2002), 970–983. <https://doi.org/10.1109/TSE.2002.1041053>
- [4] G. Antoniol, J. Cleland-Huang, J. H. Hayes, and M. Vierhauser. 2017. Grand Challenges of Traceability: The Next Ten Years. arXiv preprint arXiv:1710.03129. <https://doi.org/10.48550/arXiv.1710.03129> arXiv:1710.03129
- [5] Chetan Arora, Mehrdad Sabetzadeh, Lionel Briand, and Frank Zimmer. 2017. Automated Extraction and Clustering of Requirements Glossary Terms. *IEEE Transactions on Software Engineering* 43, 10 (2017), 918–945. <https://doi.org/10.1109/TSE.2016.2635134>
- [6] Chetan Arora, Mehrdad Sabetzadeh, Arda Göknıl, Lionel C. Briand, and Frank Zimmer. 2015. Change Impact Analysis for Natural Language Requirements: An NLP Approach. In *IEEE International Requirements Engineering Conference (RE)*. IEEE, 6–15. <https://doi.org/10.1109/RE.2015.7320403>
- [7] Alberto Bacchelli, Andrea Mocchi, Anthony Cleve, and Michele Lanza. 2017. Mining Structured Data in Natural Language Artifacts with Island Parsing. *Science of Computer Programming* 150 (2017), 31–55. <https://doi.org/10.1016/j.scico.2017.06.009>
- [8] Muneera Bano, Rashina Hoda, Didar Zowghi, and Christoph Treude. 2024. Large language models for qualitative research in software engineering: exploring opportunities and challenges. *Automated Software Engineering* 31, 1, Article 8 (2024). <https://doi.org/10.1007/s10515-023-00407-8>
- [9] Gabriele Bavota, Andrea De Lucia, Rocco Oliveto, and Genoveffa Tortora. 2014. Enhancing software artefact traceability recovery processes with link count information. *Information and Software Technology* 56, 2 (2014), 163–182. <https://doi.org/10.1016/j.infsof.2013.08.004>
- [10] Antonio Bucchiarone, Davide Di Ruscio, Henry Muccini, and Patrizio Pelliccione. 2009. From Requirements to Java Code: An Architecture-Centric Approach for Producing Quality Systems. In *Model-Driven Software Development: Integrating Quality Assurance*. IGI Global, 263–301. <https://doi.org/10.4018/978-1-60566-006-6.ch011>
- [11] N. D. Q. Bui, Y. Yu, and L. Jiang. 2021. Self-Supervised Contrastive Learning for Code Retrieval and Summarization via Semantic-Preserving Transformations. In *ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. ACM, New York, NY, USA, 511–521. <https://doi.org/10.1145/3404835.3462840>
- [12] Rafael Capilla, Juan C. Dueñas, and Francisco Nava. 2010. Viability for codifying and documenting architectural design decisions with tool support. *Journal of Software Maintenance and Evolution: Research and Practice* 22, 2 (2010), 81–119. <https://doi.org/10.1002/smr.419>
- [13] Rafael Capilla, Anton Jansen, Antony Tang, Paris Avgeriou, and Muhammad Ali Babar. 2016. 10 years of software architecture knowledge management: Practice and future. *Journal of Systems and Software* 116 (2016), 191–205. <https://doi.org/10.1016/j.jss.2015.08.054>
- [14] Rafael Capilla, Elisa Yumi Nakagawa, Uwe Zdun, and Carlos Carrillo. 2017. Toward Architecture Knowledge Sustainability: Extending System Longevity. *IEEE Software* 34, 2 (2017), 108–111. <https://doi.org/10.1109/ms.2017.54>
- [15] R. Capilla, O. Zimmermann, U. Zdun, P. Avgeriou, and J. M. Küster. 2011. An Enhanced Architectural Knowledge Metamodel Linking Architectural Design Decisions to other Artifacts in the Software Engineering Lifecycle. In *European Conference on Software Architecture (ECSA)*. Springer Berlin Heidelberg, Berlin, Heidelberg, 303–318. https://doi.org/10.1007/978-3-642-23798-0_33
- [16] J. Chen, X. Hu, Z. Li, C. Gao, X. Xia, and D. Lo. 2024. Code Search is All You Need? Improving Code Suggestions with Code Search. In *IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, New York, NY, USA, 1–13. <https://doi.org/10.1145/3597503.3639085>
- [17] Jane Cleland-Huang, Orlena Gotel, and Andrea Zisman (Eds.). 2012. *Software and Systems Traceability*. Springer, London. <https://doi.org/10.1007/978-1-4471-2239-5>
- [18] Andrea De Lucia, Fausto Fasano, Rocco Oliveto, and Genoveffa Tortora. 2007. Recovering Traceability Links in Software Artifact Management Systems Using Information Retrieval Methods. *ACM Transactions on Software Engineering and Methodology* 16, 4, Article 13 (2007). <https://doi.org/10.1145/1276933.1276934>
- [19] Andrea De Lucia, Rocco Oliveto, and Genoveffa Tortora. 2009. Assessing IR-Based Traceability Recovery Tools through Controlled Experiments. *Empirical Software Engineering* 14, 1 (2009), 57–92. <https://doi.org/10.1007/s10664-008-9090-8>
- [20] A. Eghbali and M. Pradel. 2024. De-Hallucinator: Mitigating LLM Hallucinations in Code Generation Tasks via Iterative Grounding. arXiv preprint arXiv:2401.01701. <https://doi.org/10.48550/arXiv.2401.01701> arXiv:2401.01701

- [21] Romina Etezadi, Sallam Abualhaija, Chetan Arora, and Lionel Briand. 2026. Classifier or prompt: A case study on legal requirements traceability. *Empirical Software Engineering* 31, 4, Article 85 (July 2026). <https://doi.org/10.1007/s10664-026-10827-1>
- [22] Saad Ezzini, Sallam Abualhaija, Chetan Arora, and Mehrdad Sabetzadeh. 2022. Automated Handling of Anaphoric Ambiguity in Requirements: A Multi-Solution Study. In *Proc. 44th IEEE/ACM Int. Conf. on Software Engineering (ICSE)*. ACM, New York, NY, USA, 187–199. <https://doi.org/10.1145/3510003.3510157>
- [23] Saad Ezzini, Sallam Abualhaija, Chetan Arora, and Mehrdad Sabetzadeh. 2023. AI-based Question Answering Assistance for Analyzing Natural-language Requirements. In *IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 1277–1289. <https://doi.org/10.1109/ICSE48619.2023.00113>
- [24] Saad Ezzini, Sallam Abualhaija, Chetan Arora, Mehrdad Sabetzadeh, and Lionel C. Briand. 2021. Using Domain-Specific Corpora for Improved Handling of Ambiguity in Requirements. In *IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 1485–1497. <https://doi.org/10.1109/ICSE43902.2021.00133>
- [25] D. Falessi, M. Di Penta, G. Canfora, and G. Cantone. 2017. Estimating the number of remaining links in traceability recovery. *Empirical Software Engineering* 22, 3 (2017), 996–1027. <https://doi.org/10.1007/s10664-016-9460-6>
- [26] Jean-Rémy Falleri, Marianne Huchard, Mathieu Lafourcade, Clémentine Nebut, Violaine Prince, and Michel Dao. 2010. Automatic Extraction of a WordNet-Like Identifier Network from Software. In *Proc. 18th IEEE Int. Conf. on Program Comprehension (ICPC)*. IEEE, 4–13. <https://doi.org/10.1109/ICPC.2010.12>
- [27] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53. <https://doi.org/10.1109/ICSE-FoSE59343.2023.00008>
- [28] Dominik Fuchß, Sophie Corallo, Jan Keim, Janek Speit, and Anne Koziolk. 2023. Establishing a Benchmark Dataset for Traceability Link Recovery Between Software Architecture Documentation and Models. In *Software Architecture. ECSA 2022 Tracks and Workshops*, Thais Batista, Tomáš Bureš, Claudia Raibulet, and Henry Muccini (Eds.). Springer International Publishing, Cham, 455–464. https://doi.org/10.1007/978-3-031-36889-9_30
- [29] Dominik Fuchß, Tobias Hey, Jan Keim, Haoyu Liu, Niklas Ewald, Tobias Thirolf, and Anne Koziolk. 2025. LiSSA: Toward Generic Traceability Link Recovery Through Retrieval-Augmented Generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1396–1408. <https://doi.org/10.1109/ICSE55347.2025.00186>
- [30] Dominik Fuchß, Haoyu Liu, Sophie Corallo, Tobias Hey, Jan Keim, Johannes von Geisau, and Anne Koziolk. 2026. Who’s Who? LLM-assisted Software Traceability with Architecture Entity Recognition. *ACM Transactions on Autonomous and Adaptive Systems* (April 2026). <https://doi.org/10.1145/3807453>
- [31] Tim Gemkow, Miro Conzelmann, Kerstin Hartig, and Andreas Vogelsang. 2018. Automatic Glossary Term Extraction from Large-Scale Requirements Specifications. In *Proc. 26th IEEE Int. Requirements Engineering Conf. (RE)*. IEEE, 412–417. <https://doi.org/10.1109/RE.2018.00052>
- [32] Orlena C. Z. Gotel and Anthony C. W. Finkelstein. 1994. An Analysis of the Requirements Traceability Problem. In *Proc. 1st Int. Conf. on Requirements Engineering (ICRE)*. IEEE Comput. Soc. Press, 94–101. <https://doi.org/10.1109/ICRE.1994.292398>
- [33] S. Haiduc, G. Bavota, A. Marcus, R. Oliveto, A. De Lucia, and T. Menzies. 2013. Automatic Query Reformulations for Text Retrieval in Software Engineering. In *International Conference on Software Engineering (ICSE)*. IEEE, 842–851. <https://doi.org/10.1109/ICSE.2013.6606630>
- [34] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 33, 8, Article 220 (2024), 79 pages. <https://doi.org/10.1145/3695988>
- [35] Matthew J. Howard, Samir Gupta, Lori Pollock, and K. Vijay-Shanker. 2013. Automatically Mining Software-Based, Semantically-Similar Words from Comment-Code Mappings. In *Proc. 10th Working Conf. on Mining Software Repositories (MSR)*. IEEE, 377–386. <https://doi.org/10.1109/MSR.2013.6624052>
- [36] X. Hu, G. Li, X. Xia, D. Lo, and Z. Jin. 2018. Deep Code Comment Generation. In *IEEE/ACM International Conference on Program Comprehension (ICPC)*. ACM, New York, NY, USA, 200–210. <https://doi.org/10.1145/3196321.3196334>
- [37] N. Jain, R. Kwiatkowski, B. Ray, M. K. Ramanathan, and V. Kumar. 2024. On Mitigating Code LLM Hallucinations with API Documentation. arXiv preprint arXiv:2407.09726. <https://doi.org/10.48550/arXiv.2407.09726> arXiv:2407.09726
- [38] Jan Keim, Sophie Corallo, Dominik Fuchß, Tobias Hey, Tobias Telge, and Anne Koziolk. 2024. Recovering Trace Links Between Software Architecture Documentation And Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE ’24)*. Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3597503.3639130>
- [39] Jan Keim, Sophie Schulz, Dominik Fuchß, Claudius Kocher, Janek Speit, and Anne Koziolk. 2021. Trace Link Recovery for Software Architecture Documentation. In *Software Architecture*, Stefan Biffl, Elena Navarro, Welf Löwe, Marjan Sirjani, Raffaella Mirandola, and Danny Weyns (Eds.). Springer International Publishing, Cham, 101–116. https://doi.org/10.1007/978-3-030-86044-8_7

- [40] Patricia Lago, Henry Muccini, and Hans van Vliet. 2009. A scoped approach to traceability management. *Journal of Systems and Software* 82, 1 (2009), 168–182. <https://doi.org/10.1016/j.jss.2008.08.026>
- [41] R. Li, P. Liang, M. Soliman, and P. Avgeriou. 2022. Understanding software architecture erosion: A systematic mapping study. *Journal of Software: Evolution and Process* 34, 3, Article e2423 (2022). <https://doi.org/10.1002/smr.2423>
- [42] J. Lin, Y. Liu, Q. Zeng, M. Jiang, and J. Cleland-Huang. 2021. Traceability Transformed: Generating More Accurate Links with Pre-Trained BERT Models. In *IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 324–335. <https://doi.org/10.1109/ICSE43902.2021.00040>
- [43] C. Liu, X. Xia, D. Lo, C. Gao, X. Yang, and J. Grundy. 2022. Opportunities and Challenges in Code Search Tools. *Comput. Surveys* 54, 9 (2022), 1–40. <https://doi.org/10.1145/3480027>
- [44] Diptee Luitel, Shabnam Hassani, and Mehrdad Sabetzadeh. 2023. Using Language Models for Enhancing the Completeness of Natural-Language Requirements. In *Requirements Engineering: Foundation for Software Quality*. Springer Nature Switzerland, Cham, 87–104. https://doi.org/10.1007/978-3-031-29786-1_7
- [45] Diptee Luitel, Shabnam Hassani, and Mehrdad Sabetzadeh. 2024. Improving requirements completeness: automated assistance through large language models. *Requirements Engineering* 29, 1 (2024), 73–95. <https://doi.org/10.1007/s00766-024-00416-3>
- [46] Patrick Mäder and Alexander Egyed. 2015. Do Developers Benefit from Requirements Traceability When Evolving and Maintaining a Software System? *Empirical Software Engineering* 20, 2 (2015), 413–441. <https://doi.org/10.1007/s10664-014-9314-z>
- [47] Anas Mahmoud and Nan Niu. 2015. On the Role of Semantics in Automated Requirements Tracing. *Requirements Engineering* 20, 3 (2015), 281–300. <https://doi.org/10.1007/s00766-013-0199-y>
- [48] Andrian Marcus and Jonathan I. Maletic. 2003. Recovering Documentation-to-Source-Code Traceability Links Using Latent Semantic Indexing. In *Proc. 25th Int. Conf. on Software Engineering (ICSE)*. IEEE, 125–135. <https://doi.org/10.1109/ICSE.2003.1201194>
- [49] Salome Maro, Anthony Anjorin, Rebekka Wohlrab, and Jan-Philipp Steghöfer. 2016. Traceability Maintenance: Factors and Guidelines. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, New York, NY, USA, 414–425. <https://doi.org/10.1145/2970276.2970314>
- [50] S. Maro, J.-P. Steghöfer, E. Knauss, J. Horkoff, R. Kasauli, R. Wohlrab, J. L. Korsgaard, F. Wartenberg, N. J. Strøm, and R. Alexandersson. 2021. Managing Traceability Information Models: Not Such a Simple Task After All? *IEEE Software* 38, 5 (2021), 101–109. <https://doi.org/10.1109/MS.2020.3020651>
- [51] C. McMillan, M. Grechanik, D. Poshvyanyk, Q. Xie, and C. Fu. 2011. Portfolio: Finding Relevant Functions and Their Usage. In *International Conference on Software Engineering (ICSE)*. ACM, New York, NY, USA, 111–120. <https://doi.org/10.1145/1985793.1985809>
- [52] C. McMillan, D. Poshvyanyk, and M. Reville. 2009. Combining textual and structural analysis of software artifacts for traceability link recovery. In *ICSE Workshop on Traceability in Emerging Forms of Software Engineering (TEFSE)*. IEEE, 41–48. <https://doi.org/10.1109/TEFSE.2009.5069582>
- [53] K. Moran, D. N. Palacio, C. Bernal-Cárdenas, D. McCrystal, D. Poshvyanyk, C. Shenefiel, and J. Johnson. 2020. Improving the Effectiveness of Traceability Link Recovery using Hierarchical Bayesian Networks. In *ACM/IEEE International Conference on Software Engineering (ICSE)*. ACM, New York, NY, USA, 873–885. <https://doi.org/10.1145/3377811.3380418>
- [54] H. Muccini, A. Bertolino, and P. Inverardi. 2004. Using Software Architecture for Code Testing. *IEEE Transactions on Software Engineering* 30, 3 (2004), 160–171. <https://doi.org/10.1109/TSE.2004.1271170>
- [55] A. T. Nguyen, T. T. Nguyen, H. A. Nguyen, and T. N. Nguyen. 2012. Multi-layered approach for recovering links between bug reports and fixes. In *ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*. ACM, New York, NY, USA, 1–11. <https://doi.org/10.1145/2393596.2393671>
- [56] M. Pradel, G. Gousios, J. Liu, and S. Chandra. 2020. TypeWriter: Neural Type Prediction with Search-based Validation. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, New York, NY, USA, 209–220. <https://doi.org/10.1145/3368089.3409715>
- [57] Abdallah Qusef, Gabriele Bavota, Rocco Oliveto, Andrea De Lucia, and David Binkley. 2013. Evaluating test-to-code traceability recovery methods through controlled experiments. *Journal of Software: Evolution and Process* 25, 11 (2013), 1167–1191. <https://doi.org/10.1002/smr.1573>
- [58] Abdallah Qusef, Gabriele Bavota, Rocco Oliveto, Andrea De Lucia, and Dave Binkley. 2014. Recovering test-to-code traceability using slicing and textual analysis. *Journal of Systems and Software* 88 (2014), 147–168. <https://doi.org/10.1016/j.jss.2013.10.019>
- [59] Balasubramaniam Ramesh and Matthias Jarke. 2001. Toward Reference Models for Requirements Traceability. *IEEE Transactions on Software Engineering* 27, 1 (2001), 58–93. <https://doi.org/10.1109/32.895989>

- [60] Mehrdad Sabetzadeh and Chetan Arora. 2025. Practical Guidelines for the Selection and Evaluation of Natural Language Processing Techniques in Requirements Engineering. In *Handbook on Natural Language Processing for Requirements Engineering*. Springer Nature Switzerland, Cham, 407–433. https://doi.org/10.1007/978-3-031-73143-3_15
- [61] Y. Shi, Y. Yin, Z. Wang, D. Lo, T. Zhang, X. Xia, Y. Zhao, and B. Xu. 2022. How to Better Utilize Code Graphs in Semantic Code Search?. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, New York, NY, USA, 722–733. <https://doi.org/10.1145/3540250.3549087>
- [62] Wen Siang Tan, Markus Wagner, and Christoph Treude. 2024. Detecting outdated code element references in software repository documentation. *Empirical Software Engineering* 29, 1, Article 5 (2024). <https://doi.org/10.1007/s10664-023-10397-6>
- [63] Christoph Treude and Margaret-Anne Storey. 2025. Generative AI and Empirical Software Engineering: A Paradigm Shift. In *IEEE/ACM International Conference on AI-powered Software (AIware)*. IEEE, 233–239. <https://doi.org/10.1109/AIware69974.2025.00033>
- [64] Rosalia Tufano, Ozren Dabić, Antonio Mastropaolo, Matteo Ciniselli, and Gabriele Bavota. 2024. Code Review Automation: Strengths and Weaknesses of the State of the Art. *IEEE Transactions on Software Engineering* 50, 2 (2024), 338–353. <https://doi.org/10.1109/TSE.2023.3348172>
- [65] Rosalia Tufano, Antonio Mastropaolo, Federica Pepe, Ozren Dabić, Massimiliano Di Penta, and Gabriele Bavota. 2024. Unveiling ChatGPT’s Usage in Open Source Projects: A Mining-based Study. In *International Conference on Mining Software Repositories (MSR)*. ACM, New York, NY, USA, 571–583. <https://doi.org/10.1145/3643991.3644918>
- [66] Wentao Wang, Nan Niu, Hui Liu, and Zhendong Niu. 2018. Enhancing Automated Requirements Traceability by Resolving Polysemy. In *IEEE International Requirements Engineering Conference (RE)*. IEEE, 40–51. <https://doi.org/10.1109/RE.2018.00-53>
- [67] Yuekun Wang, Lili Quan, Xiaofei Xie, Junjie Wang, and Jianjun Chen. 2025. Spec2Code: Mapping Protocol Specification to Function-Level Code Implementation. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2096–2108. <https://doi.org/10.1109/ASE63991.2025.00174>
- [68] Rebekka Wohlrab, Eric Knauss, Jan-Philipp Steghöfer, Salome Maro, Anthony Anjorin, and Patrizio Pelliccione. 2020. Collaborative Traceability Management: A Multiple Case Study from the Perspectives of Organization, Process, and Culture. *Requirements Engineering* 25, 1 (2020), 21–45. <https://doi.org/10.1007/s00766-018-0306-1>
- [69] Tao Xiao, Sebastian Baltes, Hideaki Hata, Christoph Treude, Raula Gaikovina Kula, Takashi Ishio, and Kenichi Matsumoto. 2023. 18 million links in commit messages: purpose, evolution, and decay. *Empirical Software Engineering* 28, 4, Article 91 (2023). <https://doi.org/10.1007/s10664-023-10325-8>
- [70] Tao Xiao, Hideaki Hata, Christoph Treude, and Kenichi Matsumoto. 2024. Generative AI for Pull Request Descriptions: Adoption, Impact, and Developer Interventions. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1043–1065. <https://doi.org/10.1145/3643773>
- [71] B. Yang, M. A. Al Mamun, J. M. Zhang, and G. Uddin. 2025. Hallucination Detection in Large Language Models with Metamorphic Relations. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 425–445. <https://doi.org/10.1145/3715735>

Table 7. Detailed per-run per-project results for ArchLinker across both backends.

System	Run	Project	doc-model (link)					doc-code (file)				worst comp.		harm. comp.	
			P	R	F ₁	F ₂	CMR	P	R	F ₁	F ₂	F ₁	F ₂	F ₁	F ₂
ArchLinker (GPT-5.6-terra)	Run 1	MS	.91	1.00	.95	.98	0.0	.85	.88	.87	.88	.72	.74	.96	.96
		TS	1.00	1.00	1.00	1.00	0.0	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
		TM	.79	.95	.86	.91	0.0	.80	.75	.77	.76	.72	.61	.81	.79
		BBB	.88	.90	.89	.90	0.0	.68	.93	.79	.87	.44	.56	.81	.84
		JR	.95	1.00	.97	.99	0.0	.97	1.00	.98	.99	.92	.97	.99	.99
	Run 2	MS	.91	1.00	.95	.98	0.0	.85	.88	.87	.88	.72	.74	.96	.96
		TS	1.00	1.00	1.00	1.00	0.0	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
		TM	.82	.95	.88	.92	0.0	.80	.74	.77	.75	.72	.61	.81	.79
		BBB	.88	.90	.89	.90	0.0	.68	.93	.78	.87	.44	.56	.81	.83
		JR	.90	1.00	.95	.98	0.0	.87	1.00	.93	.97	.89	.95	.97	.99
	Run 3	MS	.91	1.00	.95	.98	0.0	.85	.88	.87	.88	.72	.74	.96	.96
		TS	1.00	1.00	1.00	1.00	0.0	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
		TM	.80	.93	.86	.90	0.0	.80	.74	.77	.75	.72	.61	.79	.77
		BBB	.87	.89	.88	.88	0.0	.68	.92	.78	.86	.44	.56	.80	.82
		JR	1.00	1.00	1.00	1.00	0.0	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
ArchLinker (GPT-5.6-luna)	Run 1	MS	.91	.94	.92	.93	0.0	.84	.80	.82	.80	.65	.65	.93	.92
		TS	.93	.96	.95	.96	0.0	.97	.96	.96	.96	.91	.86	.97	.97
		TM	.62	.95	.75	.86	0.0	.77	.84	.80	.82	.30	.47	.66	.73
		BBB	.79	.81	.80	.80	0.0	.69	.83	.75	.80	.50	.56	.78	.80
		JR	.95	1.00	.97	.99	0.0	1.00	1.00	1.00	1.00	.80	.91	.96	.98
	Run 2	MS	.91	.94	.92	.93	0.0	.84	.80	.82	.80	.65	.65	.93	.92
		TS	.96	.96	.96	.96	0.0	1.00	.96	.98	.97	.91	.86	.98	.97
		TM	.72	.91	.81	.87	0.0	.79	.80	.80	.80	.50	.61	.75	.77
		BBB	.91	.82	.86	.84	0.0	.77	.91	.84	.88	.67	.56	.85	.84
		JR	1.00	1.00	1.00	1.00	0.0	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
	Run 3	MS	.91	.97	.94	.96	0.0	.84	.83	.84	.83	.65	.65	.94	.94
		TS	.93	1.00	.96	.99	0.0	.97	1.00	.99	.99	.92	.97	.99	.99
		TM	.63	.91	.75	.84	0.0	.75	.78	.77	.78	.44	.61	.70	.75
		BBB	.88	.90	.89	.90	0.0	.71	.93	.81	.87	.50	.71	.85	.89
		JR	.90	1.00	.95	.98	0.0	.89	1.00	.94	.98	.80	.91	.94	.98

A DETAILED RESULTS

We run two language-model backends, GPT-5.6-terra and GPT-5.6-luna, three runs each: both ArchLinker and the re-run Artemis baseline are stochastic. The body reports GPT-5.6-terra and the five-project, three-run average.

A.1 RQ1 and RQ2: Comparison and Architecture-Driven Metrics

Table 7 gives the per-run per-project breakdown for ArchLinker on both backends. Table 8 gives the corresponding Artemis results, together with the released GPT-5.4 and deterministic TransArc reference results.

Table 8. Detailed per-run per-project results for Artemis across both backends, with released GPT-5.4 and deterministic TransArc reference results.

System	Run	Project	doc-model (link)					doc-code (file)				worst comp.		harm. comp.	
			P	R	F ₁	F ₂	CMR	P	R	F ₁	F ₂	F ₁	F ₂	F ₁	F ₂
Artemis (GPT-5.6-terra)	Run 1	MS	.97	.97	.97	.97	0.0	.98	.93	.96	.94	.67	.83	.94	.96
		TS	.57	.96	.71	.84	0.0	.57	.97	.72	.85	.43	.65	.66	.82
		TM	1.00	.65	.79	.70	8.8	1.00	.58	.74	.64	.60	.48	.73	.63
		BBB	.78	.69	.73	.71	0.0	.64	.86	.74	.81	.33	.28	.68	.67
		JR	.80	.89	.84	.87	11.1	.79	1.00	.88	.95	.00	.00	.00	.00
	Run 2	MS	.86	.97	.91	.94	0.0	.89	.93	.91	.92	.67	.83	.94	.96
		TS	.57	1.00	.73	.87	0.0	.58	1.00	.73	.87	.43	.65	.67	.84
		TM	.89	.58	.70	.62	17.5	.95	.56	.71	.62	.00	.00	.00	.00
		BBB	.74	.65	.69	.66	0.0	.63	.85	.72	.80	.33	.28	.66	.66
		JR	.90	1.00	.95	.98	0.0	.97	1.00	.98	.99	.80	.91	.95	.98
	Run 3	MS	.88	.97	.92	.95	0.0	.90	.93	.92	.93	.92	.88	.99	.99
		TS	.58	.96	.72	.85	0.0	.58	.97	.73	.86	.43	.65	.67	.82
		TM	.87	.70	.78	.73	8.8	.83	.68	.75	.70	.62	.50	.75	.68
		BBB	.89	.76	.82	.78	3.2	.84	.83	.83	.83	.00	.00	.00	.00
		JR	.78	1.00	.88	.95	0.0	.79	1.00	.88	.95	.80	.91	.88	.95
Artemis (GPT-5.6-luna)	Run 1	MS	1.00	.97	.98	.97	0.0	1.00	.93	.96	.94	.92	.88	.99	.99
		TS	.57	1.00	.73	.87	0.0	.56	1.00	.72	.87	.43	.65	.67	.83
		TM	.46	.72	.56	.64	8.8	.58	.73	.64	.69	.57	.60	.67	.74
		BBB	.65	.66	.66	.66	0.0	.57	.86	.68	.78	.33	.28	.62	.65
		JR	.86	1.00	.92	.97	0.0	.94	1.00	.97	.99	.80	.91	.94	.97
	Run 2	MS	1.00	.94	.97	.95	0.0	1.00	.86	.93	.89	.83	.76	.98	.97
		TS	.57	1.00	.73	.87	0.0	.58	1.00	.74	.88	.43	.65	.67	.83
		TM	.51	.70	.59	.65	0.0	.71	.66	.68	.67	.57	.45	.67	.69
		BBB	.87	.66	.75	.69	3.2	.84	.76	.80	.78	.00	.00	.00	.00
		JR	.75	1.00	.86	.94	0.0	.71	1.00	.83	.92	.73	.87	.83	.93
	Run 3	MS	.97	.97	.97	.97	0.0	.93	.93	.93	.93	.86	.86	.98	.98
		TS	.57	1.00	.73	.87	0.0	.58	1.00	.74	.88	.43	.65	.67	.83
		TM	.59	.65	.62	.64	8.8	.80	.57	.66	.60	.52	.43	.68	.67
		BBB	.77	.65	.70	.67	0.0	.74	.85	.79	.82	.25	.25	.65	.65
		JR	.86	1.00	.92	.97	0.0	.89	1.00	.94	.98	.80	.91	.91	.96
Artemis (GPT-5.4)	-	MS	1.00	.87	.93	.89	0.0	1.00	.78	.88	.82	.73	.62	.94	.91
		TS	.95	.74	.83	.78	0.0	.99	.71	.83	.75	.67	.56	.83	.77
		TM	.77	.70	.73	.71	8.8	.86	.67	.75	.70	.66	.56	.76	.68
		BBB	.93	.61	.74	.66	3.2	.85	.74	.79	.76	.00	.00	.00	.00
		JR	1.00	.89	.94	.91	11.1	1.00	1.00	1.00	1.00	.00	.00	.00	.00
TransArc [†]		MS	.94	.55	.69	.60	35.5	.96	.42	.59	.48	.00	.00	.00	.00
		TS	1.00	.74	.85	.78	0.0	1.00	.71	.83	.75	.67	.56	.84	.77
		TM	.60	.86	.71	.79	0.0	.75	.90	.82	.87	.54	.65	.76	.82
		BBB	.90	.71	.79	.74	0.0	.82	.84	.83	.84	.57	.45	.84	.80
		JR	.90	1.00	.95	.98	0.0	.89	1.00	.94	.98	.80	.91	.94	.98

[†]The doc-model columns for TransArc are SWATTR, its deterministic doc-model stage (TransArc has no standalone doc-model system).

Table 9. RQ3 judging configurations by run and backend.

Backend	Run	Judges	rejects		keeps		doc-model		doc-code		
			FP	TP	TP	FP	Prec./Rec.; F ₁ /F ₂	CMR%	Prec./Rec.; F ₁ /F ₂	worst F ₁ /F ₂	harm. F ₁ /F ₂
GPT-5.6-terra	Run 1	Full	143	9	186	26	.91/.97; .94/.96	0.0	.86/.91; .88/.90	.76/.77	.91/.92
		name judge	99	9	171	23	.74/1.00; .84/.92	0.0	.76/.96; .84/.90	.56/.70	.82/.90
		coreference judge	53	1	36	3	.76/.97; .85/.92	0.0	.76/.92; .83/.88	.63/.75	.84/.89
		No judge	0	0	195	164	.67/1.00; .78/.89	0.0	.71/.96; .80/.89	.50/.67	.77/.87
	Run 2	Full	147	9	186	25	.90/.97; .93/.96	0.0	.84/.91; .87/.89	.75/.77	.91/.91
		name judge	99	9	171	23	.74/1.00; .84/.92	0.0	.76/.96; .84/.90	.56/.70	.82/.90
		coreference judge	55	1	31	2	.77/.97; .85/.92	0.0	.76/.92; .83/.88	.68/.77	.87/.90
		No judge	0	0	195	169	.67/1.00; .79/.89	0.0	.72/.96; .81/.89	.55/.70	.79/.88
	Run 3	Full	143	11	184	24	.92/.96; .94/.95	0.0	.87/.91; .88/.90	.78/.78	.91/.91
		name judge	99	11	169	23	.75/1.00; .84/.92	0.0	.77/.96; .84/.90	.56/.70	.82/.90
		coreference judge	51	2	36	1	.77/.97; .86/.92	0.0	.78/.92; .84/.88	.68/.76	.86/.89
		No judge	0	0	195	165	.67/1.00; .79/.89	0.0	.71/.96; .81/.89	.55/.70	.78/.88
	Avg	Full	144.3	9.7	185.3	25	.91/.97; .94/.95	0.0	.86/.91; .88/.90	.76/.78	.91/.91
		name judge	99	9.7	170.3	23	.75/1.00; .84/.92	0.0	.76/.96; .84/.90	.56/.70	.82/.90
		coreference judge	53	1.3	34.3	2	.76/.97; .85/.92	0.0	.77/.92; .83/.88	.66/.76	.85/.89
		No judge	0	0	195	166	.67/1.00; .79/.89	0.0	.71/.96; .81/.89	.53/.69	.78/.88
GPT-5.6-luna	Run 1	Full	192	17	177	52	.84/.93; .88/.91	0.0	.85/.88; .87/.88	.63/.69	.86/.88
		name judge	73	16	163	49	.74/.99; .83/.91	0.0	.76/.95; .84/.90	.55/.69	.81/.89
		coreference judge	135	5	55	5	.62/.95; .74/.84	0.0	.63/.91; .74/.83	.54/.69	.75/.84
		No judge	0	0	194	233	.59/1.00; .72/.85	0.0	.60/.96; .72/.84	.50/.68	.71/.84
	Run 2	Full	198	19	176	29	.90/.93; .91/.92	0.0	.88/.89; .89/.89	.75/.74	.90/.90
		name judge	95	18	162	27	.75/.99; .84/.92	0.0	.77/.95; .84/.90	.56/.69	.82/.90
		coreference judge	119	10	45	2	.67/.97; .78/.88	0.0	.66/.92; .76/.84	.57/.70	.78/.86
		No judge	0	0	195	222	.61/1.00; .74/.86	0.0	.60/.96; .73/.85	.50/.68	.72/.85
	Run 3	Full	196	12	183	45	.85/.96; .90/.93	0.0	.83/.91; .87/.89	.66/.77	.88/.91
		name judge	81	12	165	42	.74/1.00; .84/.92	0.0	.76/.96; .84/.90	.55/.69	.82/.90
		coreference judge	129	2	50	4	.65/.96; .76/.86	0.0	.63/.92; .74/.84	.56/.69	.77/.86
		No judge	0	0	195	231	.61/1.00; .73/.86	0.0	.61/.96; .73/.85	.50/.68	.73/.85
	Avg	Full	195.3	16	178.7	42	.86/.94; .90/.92	0.0	.86/.90; .87/.89	.68/.73	.88/.90
		name judge	83	15.3	163.3	39.3	.74/.99; .83/.92	0.0	.76/.95; .84/.90	.56/.69	.82/.89
		coreference judge	127.7	5.7	50	3.7	.65/.96; .76/.86	0.0	.64/.92; .75/.84	.56/.70	.76/.85
		No judge	0	0	194.7	228.7	.60/1.00; .73/.86	0.0	.60/.96; .73/.84	.50/.68	.72/.84

A.2 RQ3: Judge Contribution

Table 9 reports the judge confusion matrix for each run and the average, on both backends.

Table 10. RQ4 ablation by project across both backends.

Backend	Variant	Project	doc-model					doc-code (file)				worst comp.		harm. comp.	
			P	R	F ₁	F ₂	CMR%	P	R	F ₁	F ₂	F ₁	F ₂	F ₁	F ₂
GPT-5.6-terra	Full	MS	.91	1.00	.95	.98	0.0	.85	.88	.87	.88	.72	.74	.96	.96
		TS	1.00	1.00	1.00	1.00	0.0	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
		TM	.81	.94	.87	.91	0.0	.80	.74	.77	.75	.72	.61	.80	.78
		BBB	.87	.90	.89	.89	0.0	.68	.93	.78	.86	.44	.56	.81	.83
		JR	.95	1.00	.97	.99	0.0	.95	1.00	.97	.99	.94	.97	.98	.99
	Full name linker only	Avg	.91	.97	.94	.95	0.0	.86	.91	.88	.90	.76	.78	.91	.91
		MS	.90	.90	.90	.90	0.0	.84	.78	.81	.79	.65	.65	.92	.90
		TS	1.00	.78	.88	.81	0.0	1.00	.74	.85	.78	.67	.56	.86	.79
		TM	.81	.87	.84	.86	0.0	.80	.71	.75	.73	.72	.61	.77	.73
		BBB	.87	.87	.87	.87	0.0	.67	.90	.77	.84	.44	.52	.79	.79
	name linker only coreference linker only	JR	.98	1.00	.99	1.00	0.0	.96	1.00	.98	.99	.96	.98	.99	1.00
		Avg	.91	.88	.90	.89	0.0	.85	.83	.83	.83	.69	.66	.86	.84
		MS	1.00	.10	.18	.12	48.4	1.00	.10	.18	.12	.00	.00	.00	.00
		TS	1.00	.27	.43	.32	19.8	1.00	.30	.46	.35	.00	.00	.00	.00
		TM	.90	.22	.34	.25	19.3	.96	.18	.30	.22	.07	.05	.14	.10
	coreference linker only	BBB	1.00	.16	.28	.19	38.7	.90	.13	.22	.15	.00	.00	.00	.00
		JR	.56	.09	.16	.11	55.6	.60	.12	.19	.14	.00	.00	.00	.00
		Avg	.89	.17	.28	.20	36.3	.89	.17	.27	.20	.01	.01	.03	.02
	No knowledge	MS	1.00	.61	.76	.66	35.5	1.00	.47	.64	.53	.00	.00	.00	.00
		TS	1.00	.75	.86	.79	18.5	1.00	.49	.65	.54	.00	.00	.00	.00
		TM	.91	.86	.88	.87	0.0	.96	.69	.80	.73	.72	.61	.80	.75
		BBB	.93	.82	.87	.84	0.0	.78	.88	.83	.86	.60	.49	.83	.80
		JR	.93	1.00	.96	.99	0.0	.89	1.00	.94	.98	.89	.95	.98	.99
GPT-5.6-luna	No knowledge	Avg	.95	.81	.87	.83	10.8	.92	.71	.77	.73	.44	.41	.52	.51
	Full	MS	.91	.95	.93	.94	0.0	.84	.81	.82	.81	.65	.65	.94	.93
		TS	.94	.98	.96	.97	0.0	.98	.97	.98	.97	.91	.90	.98	.98
		TM	.66	.92	.77	.85	0.0	.77	.81	.79	.80	.41	.56	.70	.75
		BBB	.86	.84	.85	.85	0.0	.72	.89	.80	.85	.56	.61	.83	.84
	Full name linker only	JR	.95	1.00	.97	.99	0.0	.96	1.00	.98	.99	.87	.94	.97	.99
		Avg	.86	.94	.90	.92	0.0	.86	.90	.87	.89	.68	.73	.88	.90
		MS	.90	.85	.87	.86	0.0	.82	.71	.76	.73	.58	.55	.89	.86
		TS	.95	.77	.85	.80	0.0	1.00	.73	.84	.77	.67	.56	.85	.79
		TM	.66	.86	.75	.81	0.0	.78	.78	.78	.78	.41	.56	.68	.71
	name linker only coreference linker only	BBB	.85	.80	.82	.81	0.0	.72	.86	.78	.83	.55	.52	.80	.80
		JR	.95	1.00	.97	.99	0.0	.96	1.00	.98	.99	.87	.94	.97	.99
		Avg	.86	.85	.85	.85	0.0	.86	.81	.83	.82	.61	.63	.84	.83
		MS	1.00	.37	.53	.42	35.5	1.00	.32	.48	.37	.00	.00	.00	.00
		TS	.95	.42	.57	.47	4.9	.96	.38	.53	.43	.11	.08	.19	.16
	coreference linker only	TM	.85	.23	.36	.27	14.0	.93	.21	.34	.25	.11	.08	.22	.16
		BBB	.96	.22	.36	.26	33.3	.68	.13	.22	.16	.00	.00	.00	.00
		JR	.33	.02	.04	.02	92.6	.33	.04	.07	.05	.00	.00	.00	.00
		Avg	.82	.25	.37	.29	36.1	.78	.22	.33	.25	.04	.03	.08	.06
	No knowledge	MS	1.00	.59	.74	.64	35.5	1.00	.45	.62	.51	.00	.00	.00	.00
		TS	1.00	.70	.82	.75	18.5	1.00	.43	.60	.49	.00	.00	.00	.00
		TM	.74	.96	.83	.90	0.0	.93	.80	.86	.83	.48	.62	.75	.77
		BBB	.91	.72	.80	.75	0.0	.79	.86	.82	.85	.56	.52	.81	.78
		JR	1.00	1.00	1.00	1.00	0.0	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
	No knowledge	Avg	.93	.79	.84	.81	10.8	.94	.71	.78	.73	.41	.43	.51	.51

A.3 RQ4: Module Ablation

Table 10 gives the per-project module ablation on both backends. Table 11, Table 12, Table 13, and Table 14 give the per-run and average ablation.

Table 11. RQ4 module ablation, run 1, both backends.

Backend	Variant	doc-model					doc-code (file)				worst comp.		harm. comp.	
		P	R	F ₁	F ₂	CMR%	P	R	F ₁	F ₂	F ₁	F ₂	F ₁	F ₂
GPT-5.6-terra	Full	.91	.97	.94	.96	0.0	.86	.91	.88	.90	.76	.77	.91	.92
	name linker only	.92	.89	.90	.89	0.0	.86	.83	.84	.83	.70	.67	.87	.85
	coreference linker only	.91	.17	.28	.20	36.5	.94	.17	.28	.20	.04	.03	.08	.06
	No knowledge	.95	.80	.86	.82	10.8	.92	.71	.77	.73	.44	.40	.52	.51
GPT-5.6-luna	Full	.84	.93	.88	.91	0.0	.85	.88	.87	.88	.63	.69	.86	.88
	name linker only	.84	.85	.84	.84	0.0	.85	.80	.82	.81	.56	.58	.81	.81
	coreference linker only	.94	.27	.40	.31	32.0	.90	.24	.36	.28	.04	.03	.08	.06
	No knowledge	.93	.77	.83	.79	10.8	.93	.70	.76	.72	.40	.45	.51	.51

Table 12. RQ4 module ablation, run 2, both backends.

Backend	Variant	doc-model					doc-code (file)				worst comp.		harm. comp.	
		P	R	F ₁	F ₂	CMR%	P	R	F ₁	F ₂	F ₁	F ₂	F ₁	F ₂
GPT-5.6-terra	Full	.90	.97	.93	.96	0.0	.84	.91	.87	.89	.75	.77	.91	.91
	name linker only	.91	.89	.89	.89	0.0	.84	.83	.82	.82	.67	.66	.86	.85
	coreference linker only	.78	.14	.24	.17	45.3	.76	.13	.21	.15	.00	.00	.00	.00
	No knowledge	.96	.81	.87	.83	10.8	.93	.70	.77	.72	.44	.40	.52	.51
GPT-5.6-luna	Full	.90	.93	.91	.92	0.0	.88	.89	.89	.89	.75	.74	.90	.90
	name linker only	.90	.85	.87	.86	0.0	.88	.82	.84	.83	.68	.65	.86	.83
	coreference linker only	.77	.23	.34	.26	39.0	.74	.20	.30	.23	.00	.00	.00	.00
	No knowledge	.92	.81	.84	.82	10.8	.95	.72	.79	.75	.39	.41	.50	.50

Table 13. RQ4 module ablation, run 3, both backends.

Backend	Variant	doc-model					doc-code (file)				worst comp.		harm. comp.	
		P	R	F ₁	F ₂	CMR%	P	R	F ₁	F ₂	F ₁	F ₂	F ₁	F ₂
GPT-5.6-terra	Full	.92	.96	.94	.95	0.0	.87	.91	.88	.90	.78	.78	.91	.91
	name linker only	.92	.88	.89	.88	0.0	.86	.82	.83	.83	.70	.65	.86	.84
	coreference linker only	.98	.19	.31	.22	27.3	.98	.20	.32	.24	.00	.00	.00	.00
	No knowledge	.95	.82	.87	.84	10.8	.92	.71	.77	.73	.45	.42	.52	.51
GPT-5.6-luna	Full	.85	.96	.90	.93	0.0	.83	.91	.87	.89	.66	.77	.88	.91
	name linker only	.85	.86	.85	.86	0.0	.84	.82	.82	.82	.60	.64	.84	.84
	coreference linker only	.74	.26	.37	.29	37.2	.69	.21	.32	.24	.09	.07	.17	.13
	No knowledge	.94	.80	.85	.82	10.8	.95	.71	.79	.74	.43	.43	.52	.52

Table 14. RQ4 module ablation, average of the three runs, both backends.

Backend	Variant	doc-model					doc-code (file)				worst comp.		harm. comp.	
		P	R	F ₁	F ₂	CMR%	P	R	F ₁	F ₂	F ₁	F ₂	F ₁	F ₂
GPT-5.6-terra	Full	.91	.97	.94	.95	0.0	.86	.91	.88	.90	.76	.78	.91	.91
	name linker only	.91	.88	.90	.89	0.0	.85	.83	.83	.83	.69	.66	.86	.84
	coreference linker only	.89	.17	.28	.20	36.3	.89	.17	.27	.20	.01	.01	.03	.02
	No knowledge	.95	.81	.87	.83	10.8	.92	.71	.77	.73	.44	.41	.52	.51
GPT-5.6-luna	Full	.86	.94	.90	.92	0.0	.86	.90	.87	.89	.68	.73	.88	.90
	name linker only	.86	.85	.85	.85	0.0	.86	.81	.83	.82	.61	.63	.84	.83
	coreference linker only	.82	.25	.37	.29	36.1	.78	.22	.33	.25	.04	.03	.08	.06
	No knowledge	.93	.79	.84	.81	10.8	.94	.71	.78	.73	.41	.43	.51	.51

B PROMPT TEMPLATES

ArchLinker uses five LLM prompts across its three stages. Placeholders are shown in {braces}.

B.1 Alias Discovery

Alias extraction.

Find all alternative names used for these components in the document.

COMPONENTS: {component names}

Find surface forms the document uses to refer to a single named component (introduced short forms, alternate names, or words of multi-word names when they alone clearly mean the full name). Reject terms whose ordinary English use dominates.

A fragment of a longer identifier is not an alias: if a term appears only as part of a compound or qualified name, do not include it.

DOCUMENT:

{document sentences, one per line}

Return JSON:

```
{"abbreviations": [{"term": "short_form", "component": "FullComponent"}],
  "synonyms": [{"term": "specific_alternative_name", "component":
  "FullComponent"}]}
```

JSON only:

Alias judge.

JUDGE: Review these component name mappings for correctness.

COMPONENTS: {component names}

PROPOSED MAPPINGS:

{proposals as JSON}

An alias is valid when the document establishes an equivalence between a phrase and a single named component. It is invalid when the phrase is generic vocabulary or identifies anything other than that one component. When uncertain, prefer APPROVE.

Return JSON, echoing each approved mapping in full:

```
{"approved": [{"term": "term1", "component": "FullComponent"}]}
```

JSON only:

B.2 named-reference route

name judge (name judge).

Validate components in a document.

COMPONENTS: {component names}

A trace link holds when a sentence makes an architectural claim about a component. A mention that says nothing further about the component still counts.

Each case gives: the expression the sentence uses, the sentence itself, supporting evidence, and the target component. The evidence describes what the expression is doing; it is not a verdict.

written: how the sentence writes the component's name. Values: exact (full catalog name written as a name), alias (document-established

alternate form), part (one word of a multi-word name), qualified name (full name inside a longer joined or dotted identifier). A shorter surface leaves more readings open but does not make the reading wrong.

Reject only on a positive ground: the sentence asserts nothing of this component because the name is doing some other job, or the sentence denies what it would otherwise say.

Some sentences use an ordinary English word that coincides with a component's name. Approve only when the sentence uses that word as the name of the component. Capitalization is evidence but not decisive.

An expression occurring only as part of a longer joined or dotted identifier names a piece of that identifier, not a participant in the sentence. An expression denoting what a component acts on or produces refers to that thing, not to the component.

{SENTENCES table, included when partial-name cases are present}

Where the sentence does not write the name in full, that a surface can name this component is not evidence that it does here.

For each case, first quote the EXACT words from the sentence the verdict rests on, then decide approve true/false.

CASES:

Case {i}: "{span}" -> {component name}

{previous sentence, if any} "{sentence text}"

Evidence: written={exact|alias|part|qualified name}

Return JSON:

```
{ "validations": [{ "case": 1, "claim": "<exact quote or none>", "approve": true } ] }
```

JSON only:

B.3 coreference route

Coreference resolver.

Resolve references (pronouns and noun phrases that refer back) to components.

COMPONENTS: {component names}

SENTENCES (the document text the cases are drawn from)

{sentence table as JSON}

For each TARGET sentence, identify any pronoun or noun phrase in THAT sentence that refers back to a listed component. Read the TARGET's context in SENTENCES. If a target sentence has no such reference, return no resolution for it. Be conservative: only include resolutions you are certain about.

Quote the referring expression first, then name the component it points to.

Case {i}:

TARGET S{sentence number}: {sentence text}

Resolve when the surrounding sentences make one component the clear antecedent. Avoid resolving when two or more equally plausible antecedents exist.

Return JSON:

```
{
  "resolutions": [
    {
      "case": 1,
      "sentence": N,
      "reference": "the server",
      "candidates": ["Name", "OtherName"],
      "component": "Name",
      "antecedent_sentence": M,
      "antecedent_text": "exact quote with component name"
    }
  ]
}
```

JSON only:

coreference judge (coreference judge).

Validate coreference links: does the referring expression in this sentence actually refer to the named component as an architectural participant?

COMPONENTS: {component names}

Approve only when the sentence contains a genuine referring expression that unambiguously points to THIS component and makes an architectural claim about it. Reject when there is no such expression or when the antecedent could equally be a different component. An expression denoting what a component acts on or produces refers to that thing, not to the component. When uncertain, reject.

For each case, first quote the EXACT words that state the architectural claim (or "none"), then state the strongest ground for rejecting (or "none"), then decide.

CASES:

Case {i}: pronoun/role-ref -> {component name}

Claimed reference: "{reference}"

Claimed antecedent (S{n}): "{antecedent text}"

{previous sentence, if any} "{sentence text}"

Return JSON:

```
{
  "validations": [
    {
      "case": 1,
      "claim": "<exact quote or none>",
      "objection": "<strongest ground to reject, or none>",
      "approve": true
    }
  ]
}
```

JSON only: